80 UNCHANGED QUESTIONS

JAVA THREADS

ZERO TO HERO

Handwritten-Style Interview Notes



Simple English · 80 answered follow-ups · topic sketches · code walkthroughs · 10 tested labs

FOUNDATION → SENIOR / ARCHITECT

Java 17 programs · Java 21 virtual-thread supplement

START HERE

Learn once. Explain clearly.

For each question, start with the 30-second answer. Then use the sketch and original example to explain the mechanism. Finish by speaking the follow-up answer aloud.

Understand → Draw → Run → Defend

Inside the guide

01	Thread foundations & task design	Q001–Q005
02	Lifecycle, start, run & join	Q006–Q010
03	Scheduling, sleep & metadata	Q011–Q015
04	Interruption & cancellation	Q016–Q020
05	Synchronization & shared invariants	Q021–Q025
06	Waiting, notification & locks	Q026–Q030
07	Deadlocks, starvation & diagnosis	Q031–Q035
08	Executors, Futures & lifecycle	Q036–Q040
09	Thread pools, queues & overload	Q041–Q045
10	Producer-consumer & coordination	Q046–Q050
11	CompletableFuture composition	Q051–Q055
12	Scheduling, fork/join & streams	Q056–Q060
13	ThreadLocal & request context	Q061–Q065
14	Virtual threads & modern concurrency	Q066–Q070
15	Coding challenges & reproducible tests	Q071–Q075
16	Senior production scenarios	Q076–Q080

After all 80 questions: quick-reference tables, 10 tested coding labs, all Java source listings, one Java 21 example, a study calendar and interview rehearsal.

One question at a time.

01 Speak the short answer

Use the simple explanation first. Avoid memorizing too much text.

02 Draw the mechanism

Recreate the source-specific flow. Explain which thread holds, waits, publishes or completes.

03 Show Java code

Read the original code example, check assumptions, then run its lab when available.

04 Answer the follow-up

Every original follow-up is answered separately, with a focus on safe operation and edge cases.

Technical baseline: The Q&A and original examples are grounded in the attached 71-page guide. The shorter explanations, follow-up answers and sketches are newly authored teaching aids. Java 17 is the main lab baseline. Java 21/25 differences must be checked on the actual runtime. A test passing does not prove every possible thread schedule is safe.

PART I

FOUNDATIONS & CORRECTNESS

Understand what a thread is, how it runs, and what shared state means.

> Task → Thread → Synchronization

QUESTIONS 001–020

Keep your explanations short; make your concurrency guarantees exact.

Thread foundations & task design

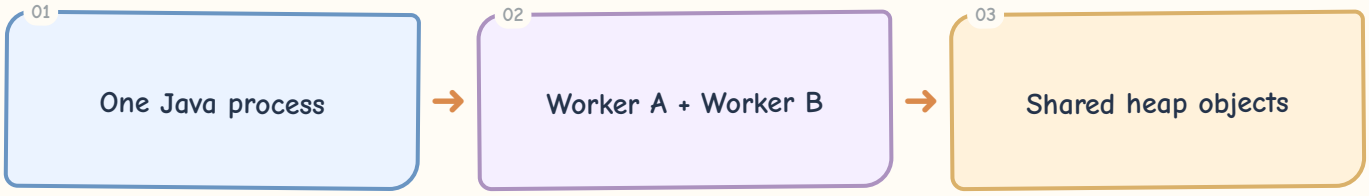
Q001. What is a thread, and how is it different from a process?

THE 30-SECOND ANSWER

A thread is one path of work inside a running program. Threads in the same Java process share objects, but each has its own call stack. Separate processes have their own memory space.

Why it matters: Two requests can reach the same mutable state concurrently.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE *original illustration*

```
One JVM process
Request thread A -> shared service object
Request thread B -> the same service object
```

INTERVIEWER'S FOLLOW-UP

Which data would you keep immutable in a shared service?

Answer → Keep request DTOs, configuration snapshots and shared lookup values immutable where possible. Build a new value when changes are needed.

! COMMON TRAP A separate stack does not make every object referenced from it private.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread foundations & task design

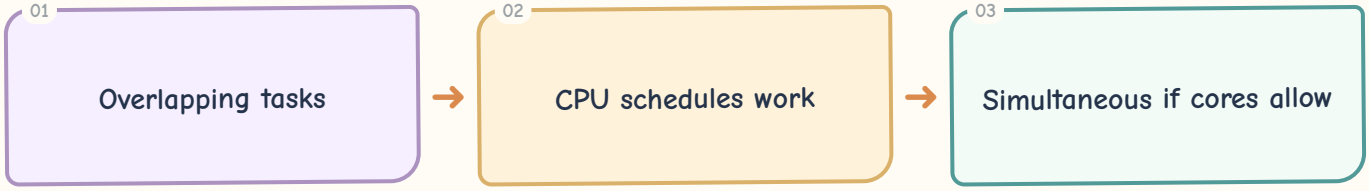
Q002. What is concurrency versus parallelism?

THE 30-SECOND ANSWER

Concurrency means tasks make progress during overlapping time periods. Parallelism means tasks actually run at the same moment on different CPU capacity. One CPU core can switch between concurrent tasks.

Why it matters: Concurrency can improve utilization when work waits, without increasing CPU capacity.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Concurrent: A waits while B makes progress
Parallel: A and B compute on different cores
More tasks != more available CPU
```

INTERVIEWER'S FOLLOW-UP

Why can adding threads slow a CPU-bound workload?

Answer → Extra runnable workers fight for the same CPU, creating context switches and cache pressure. If all cores are busy, throughput may fall.

COMMON TRAP Thread count is not a measure of useful parallel throughput.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread foundations & task design

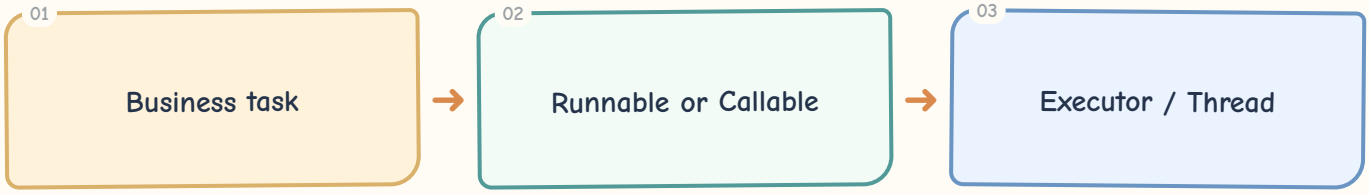
Q003. Should you extend Thread or implement Runnable?

THE 30-SECOND ANSWER

Put the task in a Runnable or Callable, then decide separately where it runs. This keeps business work reusable and avoids tying it to a particular Thread object.

Why it matters: The same task abstraction can later be submitted to an executor.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Runnable work = () -> processOrder();
Thread thread = new Thread(work, "order-worker");
thread.start();
```

INTERVIEWER'S FOLLOW-UP

How does Callable change the task contract?

Answer → Callable<V> returns a typed result and can throw checked exceptions. An executor normally exposes this through Future<V>.

! COMMON TRAP Reusing a task object is safe only if its mutable state supports that reuse.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread foundations & task design

Q004. How do Runnable and Callable differ?

THE 30-SECOND ANSWER

Runnable performs work without returning a value. Callable returns a value and may declare checked exceptions. An executor runs either task and can give you a Future for its result.

Why it matters: The Future represents the submitted execution and its result or failure.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
Callable<Integer> work = () -> 42;
Future<Integer> result = pool.submit(work);
int value = result.get();
```

INTERVIEWER'S FOLLOW-UP

What does submit(Runnable) return on successful completion?

Answer → submit(Runnable) returns Future<?>. After successful completion, its get() returns null; failures appear through ExecutionException.

COMMON TRAP A Runnable can still fail with an unchecked exception.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread foundations & task design

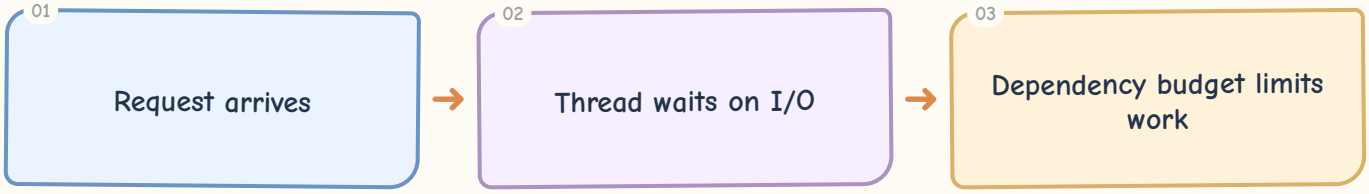
Q005. When is a thread-per-request design appropriate?

THE 30-SECOND ANSWER

One thread per request is easy to understand when load and downstream resources are limited. Platform threads are relatively costly; virtual threads can make blocking waits cheaper, but database connections still need limits.

Why it matters: The bottleneck may be the dependency rather than the number of runnable tasks.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Request -> execution resource -> database connection
Thread choice and connection budget are separate decisions.
Bound admission when the dependency cannot keep up.
```

INTERVIEWER'S FOLLOW-UP

When would asynchronous I/O fit better than blocking calls?

Answer → Use async I/O when there are many long waits, a small controlled thread budget and good asynchronous client support. Measure its complexity.

⚠ COMMON TRAP Cheap threads do not make queues, connections or payload memory unlimited.

MY ONE-LINE RECAP _____

Lifecycle, start, run & join

Q006. What is the difference between start and run?

THE 30-SECOND ANSWER

Calling start() launches a new thread that runs the task. Calling run() directly on a conventional platform Thread executes an ordinary method call on the current thread.

Why it matters: The direct call and the started execution are two task invocations in this example.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Thread t = new Thread(() -> use(Thread.currentThread()));
t.run();   // conventional platform task: caller executes
t.start(); // new thread executes its task
```

INTERVIEWER'S FOLLOW-UP

Which thread would an exception from the direct call reach?

Answer → It reaches the caller of run(), because no new thread boundary was created by that direct call.

! COMMON TRAP Java 25 Thread.run documentation has special behavior for virtual threads; start them normally.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Lifecycle, start, run & join

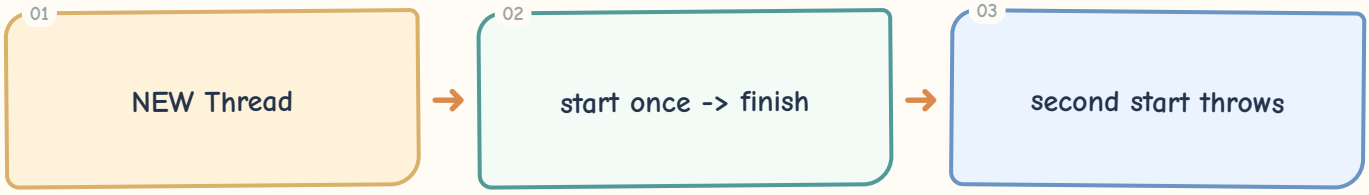
Q007. Can a Thread be started twice or restarted after termination?

THE 30-SECOND ANSWER

A Thread object can start only once. After it ends, create another Thread or submit another task to an executor. You may reuse task code only when its state is safe.

Why it matters: Completion does not reset the Thread object's start eligibility.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Thread t = new Thread(work);
t.start(); t.join();
t.start(); // IllegalThreadStateException
```

INTERVIEWER'S FOLLOW-UP

What state makes a Runnable unsafe to submit twice concurrently?

Answer → Shared mutable fields such as counters or iteration indexes can race between submissions. Use stateless tasks or coordinate changes.

! COMMON TRAP A reusable Runnable and a reusable Thread are different things.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Lifecycle, start, run & join

Q008. What are the six Java thread states?

THE 30-SECOND ANSWER

Java reports NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED. These are snapshots of the JVM's view, not a promise about which thread will execute next.

Why it matters: A state snapshot is evidence about a moment, not a whole causal history.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE *original illustration*

```
NEW -> start -> RUNNABLE
Monitor contention -> BLOCKED
Completion of run -> TERMINATED
```

INTERVIEWER'S FOLLOW-UP

Why might RUNNABLE not mean that a thread is consuming CPU?

Answer → RUNNABLE also covers eligible threads waiting for CPU time or some OS operations. It is not an observed CPU-usage metric.

⚠ COMMON TRAP RUNNING is not a separate Thread.State enum constant.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Lifecycle, start, run & join

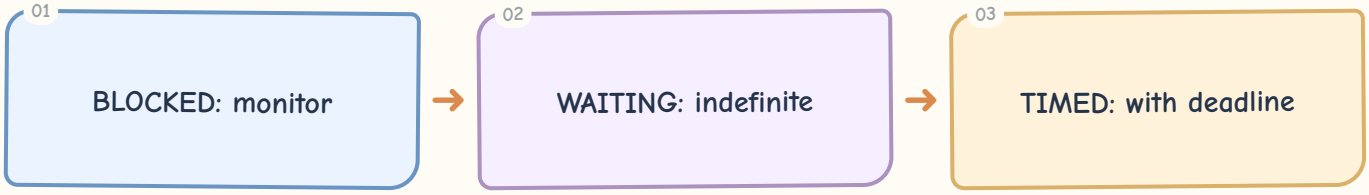
Q009. How do BLOCKED, WAITING and TIMED_WAITING differ?

THE 30-SECOND ANSWER

BLOCKED usually means waiting to enter a synchronized monitor. WAITING means an untimed wait or park; TIMED_WAITING has a timeout. Read the stack trace to find the real cause.

Why it matters: After notification, an Object.wait waiter may become BLOCKED while reacquiring its monitor.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Monitor entry: BLOCKED
Untimed Object.wait: WAITING while waiting
Thread.sleep: TIMED_WAITING
```

INTERVIEWER'S FOLLOW-UP

What stack evidence would distinguish a pool queue wait from monitor contention?

Answer → Queue wait often shows LockSupport.park in ThreadPoolExecutor.getTask; a monitor wait shows BLOCKED and an owned monitor in the dump.

! COMMON TRAP Every lock wait is not represented by BLOCKED.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Lifecycle, start, run & join

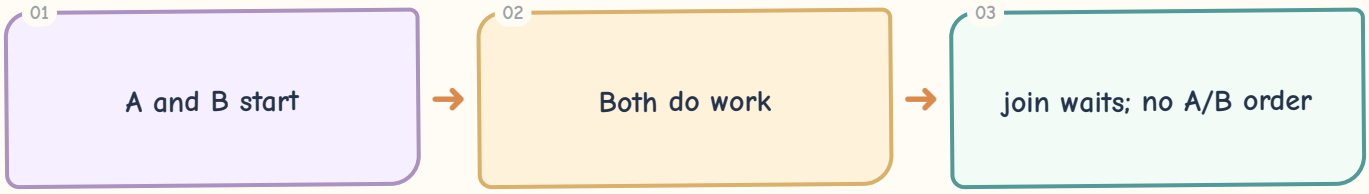
Q010. What does join guarantee, and does it order two workers?

THE 30-SECOND ANSWER

join() lets one thread wait until another finishes, and then safely observe that worker's completed actions. Calling join on A before join on B does not force A and B to run in that order.

Why it matters: The observer can process completed results without asserting worker print order.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
a.start(); b.start();
a.join(); b.join();
// Both finished; no promise that a ran before b.
```

INTERVIEWER'S FOLLOW-UP

How would you require B to start only after A completes?

Answer → Start B only after A.join() succeeds, or have A signal a latch/future that starts the dependent B task.

! COMMON TRAP Joining A before B is not equivalent to starting B after A finishes.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, sleep & metadata

Q011. Does sleep release a lock?

THE 30-SECOND ANSWER

sleep() pauses execution but keeps every lock the thread owns. Object.wait() releases its associated monitor while waiting and takes it again before returning.

Why it matters: Other threads needing this monitor must wait through the sleep.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
synchronized (lock) {
Thread.sleep(100); // still owns lock
}
```

INTERVIEWER'S FOLLOW-UP

Why might a timed Condition await be a better design here?

Answer → Condition.await releases its associated ReentrantLock while waiting and can return on timeout. It avoids holding a lock during idle time.

COMMON TRAP sleep and wait are not interchangeable delay methods.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, sleep & metadata

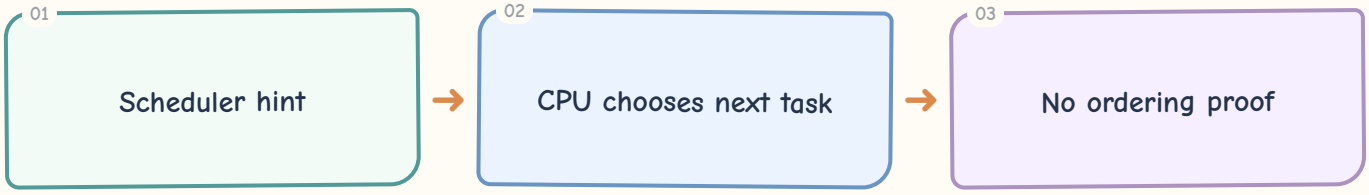
Q012. Does yield or a higher priority guarantee execution order?

THE 30-SECOND ANSWER

yield() and thread priority are hints to the scheduler. They do not promise timing or order. If one task must happen before another, use an explicit handoff.

Why it matters: Correctness must survive any scheduling allowed by the contract.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
A: Thread.yield();
B: ready to run
// No guarantee B runs next.
```

INTERVIEWER'S FOLLOW-UP

How would you test a starvation hypothesis?

Answer → Track the oldest waiting time, per-thread progress and repeated stack snapshots under load. Check which worker never gains useful CPU/resource access.

COMMON TRAP Priority is not a substitute for a dependency graph or service deadline.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, sleep & metadata

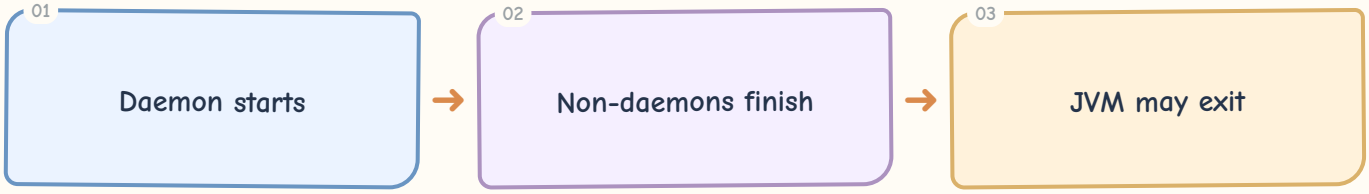
Q013. What are daemon threads?

THE 30-SECOND ANSWER

A daemon thread does not keep the JVM alive by itself. The JVM may shut down while daemon work remains, so important writes must have a controlled shutdown path.

Why it matters: The application must explicitly coordinate completion when background work must finish.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
Thread worker = new Thread(backgroundTask);
worker.setDaemon(true);
worker.start();
```

INTERVIEWER'S FOLLOW-UP

How would you drain a background writer during shutdown?

Answer → Stop taking new messages, signal the writer to finish its queue, wait with a deadline and confirm the final flush; report anything not written.

COMMON TRAP Daemon status does not guarantee finally blocks run before process termination.

MY ONE-LINE RECAP _____

Scheduling, sleep & metadata

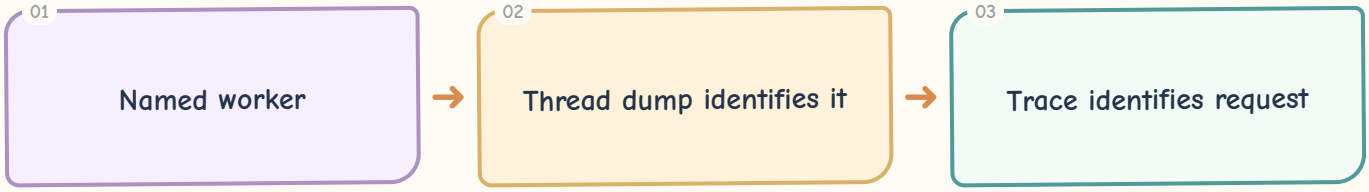
Q014. Why should worker threads be named?

THE 30-SECOND ANSWER

Name long-lived worker threads after their job, such as billing-worker-3. Good names help you match thread dumps with logs; request IDs belong in separate tracing context.

Why it matters: A dump can associate waiting workers with the billing workload.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
ThreadFactory factory = task ->
new Thread(task, "billing-worker-" + sequence.incrementAndGet());
// sequence is an AtomicInteger.
```

INTERVIEWER'S FOLLOW-UP

Which request context belongs in an explicit tracing scope instead?

Answer → Store request ID, trace/span IDs and tenant context in a bounded tracing scope, not in a long-lived thread name.

⚠ COMMON TRAP Reused pool-thread names are not unique request identifiers.

MY ONE-LINE RECAP _____

Scheduling, sleep & metadata

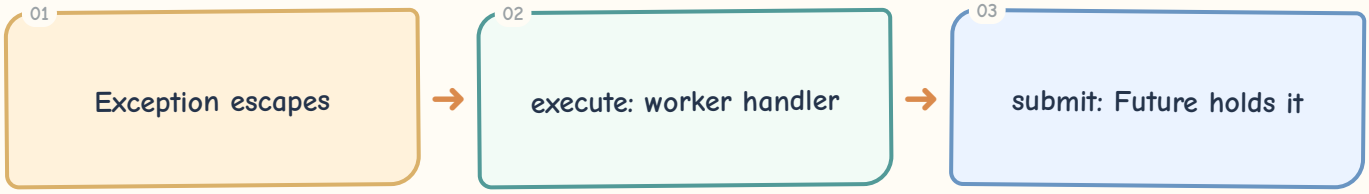
Q015. What does an UncaughtExceptionHandler handle?

THE 30-SECOND ANSWER

An UncaughtExceptionHandler can report an exception that escapes a thread. When work is submitted with submit(), the exception may instead be stored in its Future, so that result must be observed.

Why it matters: Failure routing depends on how the task was executed.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
t.setUncaughtExceptionHandler((thread, error) -> report(error));
Future<?> f = pool.submit(() -> { throw new RuntimeException(); });
// f.get() exposes this submitted failure through ExecutionException.
```

INTERVIEWER'S FOLLOW-UP

How do execute and submit differ in exception observation?

Answer → execute can let an exception escape the worker and reach its handler. submit typically captures failure in the returned Future; call get or report it.

! COMMON TRAP A handler logs a failure; it does not generally recover the failed work or transaction.

MY ONE-LINE RECAP _____

Interruption & cancellation

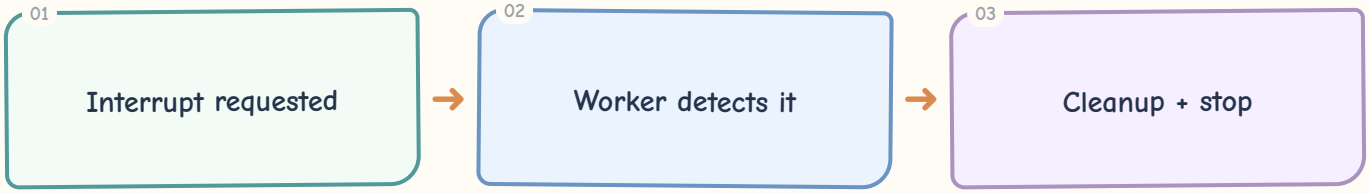
Q016. What does interrupt actually do?

THE 30-SECOND ANSWER

interrupt() asks a worker to stop cooperatively. A blocked method may throw InterruptedException, or running code may need to check its interrupt flag. It is not a kill switch.

Why it matters: The worker must reach a check or an interruptible operation.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
while (!Thread.currentThread().isInterrupted()) {
doBoundedWork();
}
```

INTERVIEWER'S FOLLOW-UP

How would you cancel a dependency call that does not react to interruption?

Answer → Give the dependency its own timeout, use supported client cancellation and close its resources. If it cannot be interrupted, isolate and bound it.

COMMON TRAP A CPU loop that never checks can ignore the request.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Interruption & cancellation

Q017. How do interrupted and isInterrupted differ?

THE 30-SECOND ANSWER

isInterrupted() checks a thread's interrupt flag without clearing it. The static Thread.interrupted() checks and clears the flag on the current thread.

Why it matters: Clearing status changes what later cancellation checks observe.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
boolean seen = Thread.currentThread().isInterrupted();
boolean consumed = Thread.interrupted();
// Second call clears current status if it was set.
```

INTERVIEWER'S FOLLOW-UP

When is consuming the status intentional?

Answer → When code intentionally acknowledges a cancellation and handles it completely. Otherwise clearing the flag may hide the request from callers.

COMMON TRAP interrupted is not a status query on an arbitrary target thread.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Interruption & cancellation

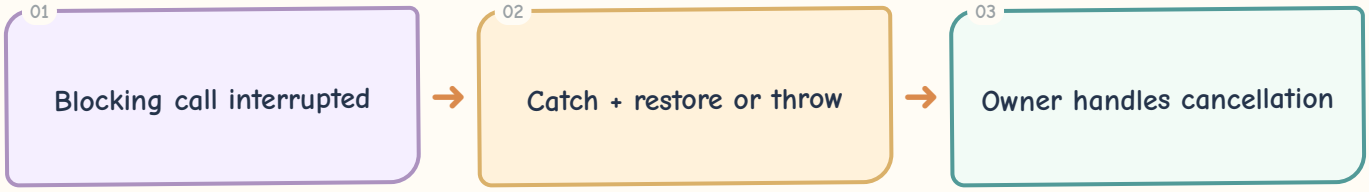
Q018. What should you do with InterruptedException?

THE 30-SECOND ANSWER

When InterruptedException occurs, either pass it to the caller or restore the interrupt flag if your method cannot pass it along. Then follow your task's cancellation policy.

Why it matters: Restoration allows outer layers to observe the cancellation request.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
catch (InterruptedException e) {
Thread.currentThread().interrupt();
return CANCELLED;
}
```

INTERVIEWER'S FOLLOW-UP

When may a top-level task consume interruption and exit without restoration?

Answer → A top-level task that owns cancellation may consume the status when it is about to exit and will not hand control to another cancellation-aware layer.

⚠ COMMON TRAP Restoring status then blindly retrying an interruptible operation can create a rapid failure loop.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Interruption & cancellation

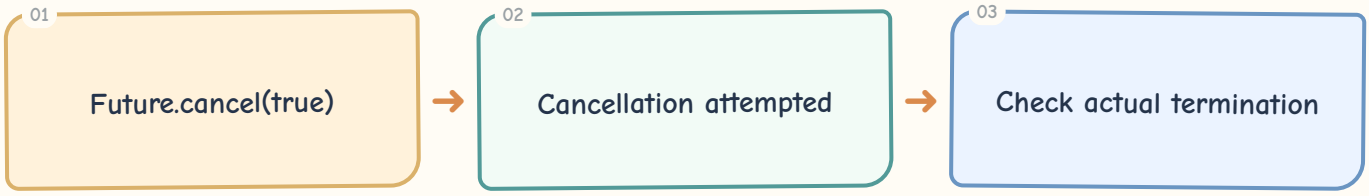
Q019. Does Future.cancel(true) prove the task stopped?

THE 30-SECOND ANSWER

Future.cancel(true) marks the Future cancelled and may interrupt the task. It does not prove the running code, network call or database write has stopped.

Why it matters: The caller must not confuse a cancelled handle with released external resources.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
boolean cancelled = future.cancel(true);
// Future cancellation != proof that remote or local work ended.
// Observe the task's supported termination boundary.
```

INTERVIEWER'S FOLLOW-UP

How would you stop a queued task versus an executing task?

Answer → Remove a queued task with its Future's cancellation path. For executing work, interrupt/cooperate and verify the actual cleanup or completion boundary.

COMMON TRAP Cancellation cannot undo a side effect that already committed.

MY ONE-LINE RECAP _____

Interruption & cancellation

Q020. Why should stop, suspend and resume be avoided?

THE 30-SECOND ANSWER

Do not use the old forceful stop, suspend or resume methods. Request cancellation, wake a blocked worker, let it finish safe cleanup, and observe actual completion.

Why it matters: The protocol can preserve application invariants at defined boundaries.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Request cancellation
Wake supported blocking operation
Cleanup in finally -> report completion
```

INTERVIEWER'S FOLLOW-UP

Which stop state and wakeup mechanism would your worker need?

Answer → Use a volatile stop flag (or interruption) plus a wakeup such as interrupt or condition.signalAll; then join/await completion.

COMMON TRAP Deprecated methods are not a fallback for unresponsive dependencies.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

PART II

EXECUTION POLICY

Choose the right locks, executors, queues, and admission limits.

> Budget → Queue → Worker → Result

QUESTIONS 021–040

Keep your explanations short; make your concurrency guarantees exact.

Synchronization & shared invariants

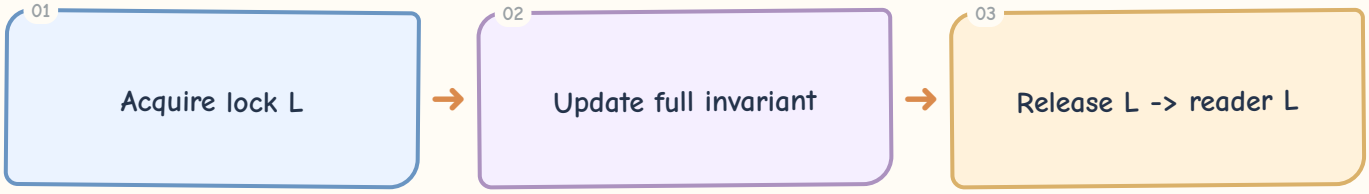
Q021. What guarantees does synchronized provide?

THE 30-SECOND ANSWER

synchronized lets one thread at a time use the same monitor and establishes ordering between matching unlock and lock. Guard every read and write that belongs to the shared rule.

Why it matters: One critical section represents the related state transition.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
synchronized (lock) { balance -= amount; ledger.add(entry); }
// Read balance and ledger together under this same lock.
```

INTERVIEWER'S FOLLOW-UP

How would you prevent future getters from bypassing the policy?

Answer → Encapsulate fields, keep them private, expose snapshot methods and state in documentation that the same lock guards every access.

! COMMON TRAP Two unrelated locks do not connect the accesses.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Synchronization & shared invariants

Q022. What locks do instance and static synchronized methods use?

THE 30-SECOND ANSWER

A synchronized instance method locks that particular object (this). A static synchronized method locks its Class object. They are different locks.

Why it matters: Identify the exact object before claiming two calls cannot overlap.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
synchronized void instanceWork() {} // this
static synchronized void classWork() {} // Example.class
// Different monitor identities.
```

INTERVIEWER'S FOLLOW-UP

What if two class loaders define the same class name?

Answer → Each defining class loader creates its own Class identity and monitor. Same binary class name does not imply the same static lock.

! COMMON TRAP Synchronizing all methods by name does not prove shared static state is protected.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Synchronization & shared invariants

Q023. What does reentrant locking mean?

THE 30-SECOND ANSWER

Reentrant means a thread may take a lock it already owns. Each lock acquisition must have a matching release. Reentrancy does not prevent deadlocks between different locks.

Why it matters: The nested call can proceed while preserving the monitor's ownership.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
synchronized void outer() { inner(); }
synchronized void inner() { update(); }
// Same receiver monitor, same thread.
```

INTERVIEWER'S FOLLOW-UP

How would recursion affect ReentrantLock hold counts?

Answer → Each successful lock adds to the hold count. Every recursive path must unlock once, normally in finally, until the count reaches zero.

COMMON TRAP Reentrancy is not a general deadlock-prevention feature.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Synchronization & shared invariants

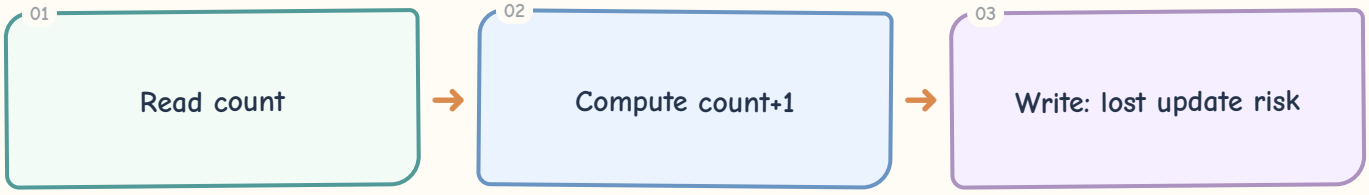
Q024. Why is volatile not enough for count++ or check-then-act?

THE 30-SECOND ANSWER

volatile makes individual reads and writes ordered, but count++ is still read, add, then write. Use AtomicInteger or lock the full check-and-update operation.

Why it matters: Visibility and business-operation atomicity are separate requirements.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
volatile int count;
count++; // read -> add -> write
// AtomicInteger.incrementAndGet is one specified atomic update.
```

INTERVIEWER'S FOLLOW-UP

How would you atomically reserve capacity and record ownership?

Answer → Represent reservation and owner in one immutable state updated through CAS, or guard both updates with one lock, including rollback on failure.

COMMON TRAP An atomic counter plus unrelated state updates may still violate a larger invariant.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Synchronization & shared invariants

Q025. How should you safely share initialized objects?

THE 30-SECOND ANSWER

Finish building an object, then share it through volatile, a lock, safe class initialization or a concurrent queue. Safe sharing of its initial state does not make later mutations safe.

Why it matters: The reader obtains initialized snapshot state through the handoff.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Config next = validateAndBuild();
current = next; // current is volatile
Reader: Config local = current;
```

INTERVIEWER'S FOLLOW-UP

Which final-field guarantees differ from a general publication edge?

Answer → Properly constructed final fields have special initialization visibility even with otherwise unsafe publication; this does not publish arbitrary later mutations.

⚠ COMMON TRAP Safely publishing an ArrayList does not make its later concurrent mutation safe.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Waiting, notification & locks

Q026. Why must wait be called while owning the monitor?

THE 30-SECOND ANSWER

Call wait() only while holding that object's monitor in a synchronized block. wait() releases that monitor, then reacquires it before returning.

Why it matters: The waiter checks state while coordinated with the publisher.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
synchronized (lock) {
while (!ready) lock.wait();
consume();
}
```

INTERVIEWER'S FOLLOW-UP

Why can holding another lock across wait create a deadlock?

Answer → wait releases only its own monitor. If the waiter still holds lock B needed by the notifier, both can remain stuck.

COMMON TRAP wait releases this monitor, not every other lock held by the thread.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Waiting, notification & locks

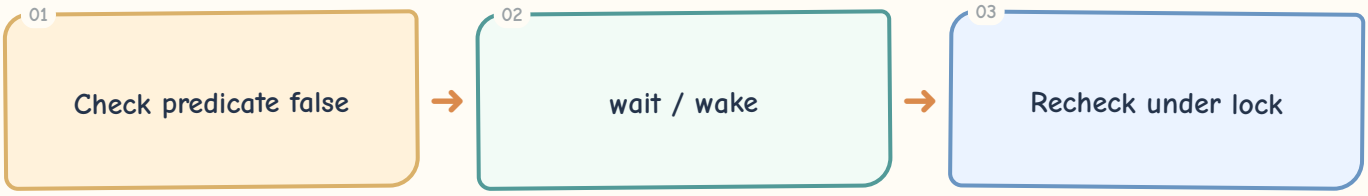
Q027. Why must waiting use while rather than if?

THE 30-SECOND ANSWER

Use while around wait or await. A thread can wake even when the needed condition is false, so it must recheck the condition under the lock.

Why it matters: The state predicate authorizes progress.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
while (queue.isEmpty()) notEmpty.await();
Item item = queue.remove();
// The same lock guards the check and removal.
```

INTERVIEWER'S FOLLOW-UP

How would you handle remaining timeout after repeated wakeups?

Answer → Compute an absolute deadline with System.nanoTime(), recalculate remaining time after every wakeup, and check the predicate before returning success.

COMMON TRAP A single notification is not a durable permission to consume.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Waiting, notification & locks

Q028. How do notify and notifyAll differ?

THE 30-SECOND ANSWER

notify() wakes one waiter; notifyAll() wakes all waiters for that monitor. They must wait to reacquire the lock, and each must recheck its condition.

Why it matters: Each awakened thread rechecks its own guarded condition.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
synchronized (lock) {
ready = true;
lock.notifyAll();
} // release lets an eligible waiter reacquire
```

INTERVIEWER'S FOLLOW-UP

When would separate Condition objects improve clarity?

Answer → Use separate notEmpty and notFull Condition queues for bounded buffers. Signal the matching condition after changing the state.

! COMMON TRAP notifyAll does not make every waiter execute simultaneously inside the critical section.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Waiting, notification & locks

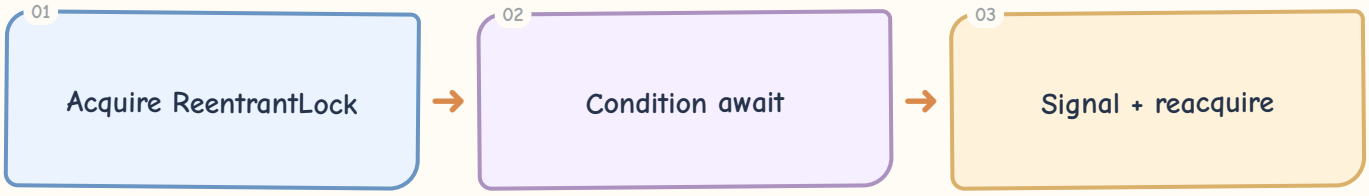
Q029. When would you choose ReentrantLock and Condition?

THE 30-SECOND ANSWER

ReentrantLock adds features such as timed or interruptible lock attempts and separate Condition queues. Always unlock in finally after acquiring it.

Why it matters: Waiting temporarily releases the associated lock and reacquires it on the specified return path.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
lock.lockInterruptibly();
try { while (!ready) condition.await(); consume(); }
finally { lock.unlock(); }
```

INTERVIEWER'S FOLLOW-UP

What does a fair ReentrantLock fail to guarantee about scheduling?

Answer → Fair locking affects acquisition policy for waiting threads, not CPU scheduling, thread deadlines or starvation outside that lock.

COMMON TRAP Do not call unlock after an acquisition that failed.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Waiting, notification & locks

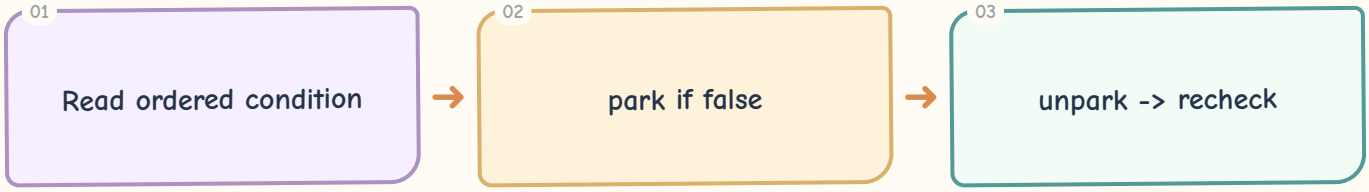
Q030. How does LockSupport differ from wait/notify?

THE 30-SECOND ANSWER

LockSupport.park() blocks a thread using a one-permit model; unpark() makes a permit available. The program still needs a safe shared condition and a loop to check it.

Why it matters: The permit supports wakeup, while the state protocol determines valid progress.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
while (!orderedReady()) LockSupport.park();
Publisher: update ordered state; unpark(worker);
// Prefer a higher-level utility when it fits.
```

INTERVIEWER'S FOLLOW-UP

Why might a hand-written parking protocol be difficult to maintain?

Answer → It must handle lost-looking signals, spurious returns, interrupt status, state visibility and ownership rules. Higher-level queues/conditions already handle this.

! COMMON TRAP Returning from park does not imply the condition became true.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Deadlocks, starvation & diagnosis

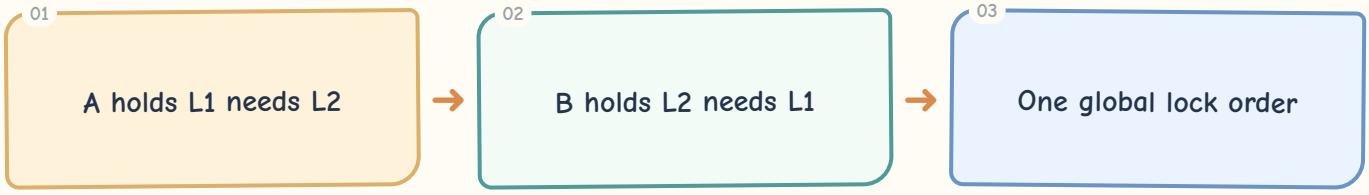
Q031. What is deadlock, and how does lock ordering prevent one class of it?

THE 30-SECOND ANSWER

Deadlock occurs when participants wait in a dependency cycle. Taking shared locks in one consistent order avoids the common A-holds-1 / B-holds-2 cycle.

Why it matters: A consistent order removes the illustrated monitor cycle.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Bad: A holds L1 -> needs L2
Bad: B holds L2 -> needs L1
Policy: both acquire L1 before L2
```

INTERVIEWER'S FOLLOW-UP

How would equal account identifiers affect a transfer lock order?

Answer → If the same account appears twice, lock it once. For different accounts sort by a unique stable identity and define a tie-breaker.

! COMMON TRAP Lock ordering alone does not fix executor or database dependency cycles.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Deadlocks, starvation & diagnosis

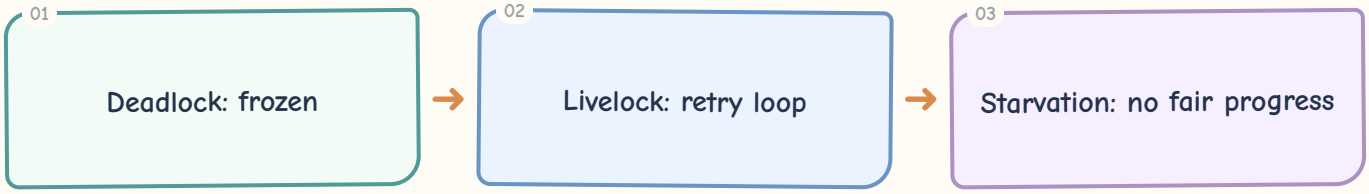
Q032. How do deadlock, livelock and starvation differ?

THE 30-SECOND ANSWER

Deadlock means no participant can move because of a cycle. Livelock means everyone keeps reacting but never finishes. Starvation means one participant keeps missing its turn.

Why it matters: The appropriate repair depends on the type of progress failure.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Deadlock: both hold a resource needed by the other
Livelock: both repeatedly retreat and retry
Starvation: one caller keeps losing access
```

INTERVIEWER'S FOLLOW-UP

How could randomized backoff help one case but not another?

Answer → Jittered retries can break two actors repeatedly colliding in a livelock. It cannot release locks in a true deadlock by itself.

⚠ COMMON TRAP High CPU does not rule out livelock; low CPU does not establish deadlock.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Deadlocks, starvation & diagnosis

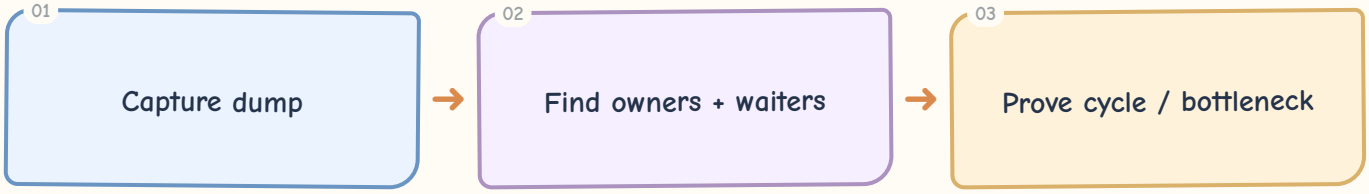
Q033. How can you detect a Java lock deadlock?

THE 30-SECOND ANSWER

Capture thread dumps and inspect which threads own locks and which are waiting. ThreadMXBean can detect certain Java lock cycles; not all resource-dependency deadlocks are covered.

Why it matters: A detected cycle is stronger evidence than a single waiting stack.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
ThreadMXBean bean = ManagementFactory.getThreadMXBean();
long[] ids = bean.findDeadlockedThreads();
// null means none detected within this API's coverage.
```

INTERVIEWER'S FOLLOW-UP

What would you collect for an executor starvation incident?

Answer → Collect pool active count, queue size and oldest age, thread dumps, blocked dependency timings and which tasks wait for child Futures.

⚠ COMMON TRAP A null result does not prove every queue, dependency or virtual-thread workflow is live.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Deadlocks, starvation & diagnosis

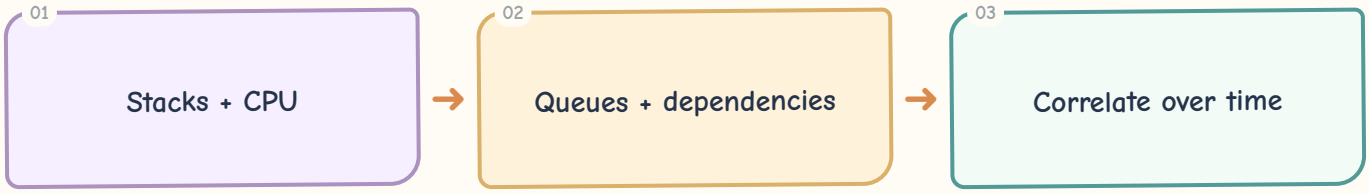
Q034. How should you read a thread dump for a slow service?

THE 30-SECOND ANSWER

Read thread dumps with queue depth, CPU use and dependency timings. Repeated BLOCKED, parked, or busy stack traces can reveal the bottleneck; a single snapshot is not enough.

Why it matters: Correlation makes the stack evidence actionable.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Evidence: repeated stacks + lock owners + CPU trend
Context: queue depth, oldest age, dependency timing
Question: which resource blocks useful completion?
```

INTERVIEWER'S FOLLOW-UP

How would you distinguish an idle pool from a saturated one?

Answer → An idle pool has waiting workers and low queue age; a saturated pool has all workers busy, growing queued work and rising completion latency.

⚠ COMMON TRAP A sleeping maintenance thread is not automatically the request bottleneck.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Deadlocks, starvation & diagnosis

Q035. Why can more threads make latency worse?

THE 30-SECOND ANSWER

Adding threads can make work slower when CPU, a lock or a database is already full. More requests then wait longer and create extra scheduling and memory cost.

Why it matters: Capacity must be understood at each resource boundary.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
100 workers -> 20 database connections
More workers may wait rather than complete more SQL.
Also inspect CPU quota and queue age.
```

INTERVIEWER'S FOLLOW-UP

Which metrics would justify reducing rather than increasing concurrency?

Answer → If completed work stays flat or drops while queue age, lock waits, DB waits and p99 rise, reduce admission/concurrency and retest.

! COMMON TRAP High thread count alone is not a root-cause diagnosis.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Executors, Futures & lifecycle

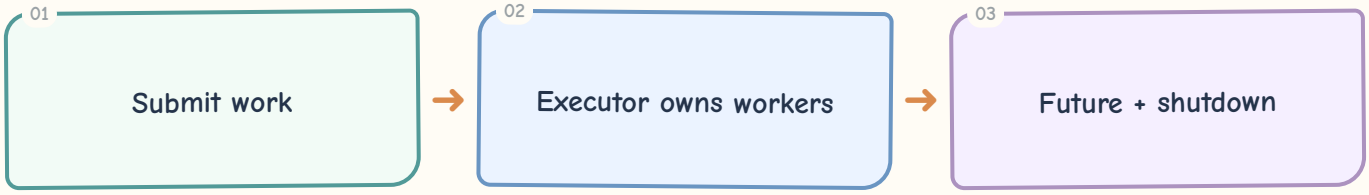
Q036. Why use ExecutorService instead of creating raw threads everywhere?

THE 30-SECOND ANSWER

ExecutorService separates task code from the policy that runs it. It manages workers and lets callers observe task results, failures and cancellation; the application must still own the pool lifecycle.

Why it matters: The application can select execution policy independently of task code.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Future<Order> f = orderPool.submit(() -> loadOrder(id));
Order order = f.get();
// The owner manages the pool's lifecycle.
```

INTERVIEWER'S FOLLOW-UP

Who should shut down a shared application executor?

Answer → The service/container lifecycle owner should shut it down exactly once at application stop, not each request or task.

COMMON TRAP Creating a new pool for every request can multiply resources and leak lifecycle ownership.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Executors, Futures & lifecycle

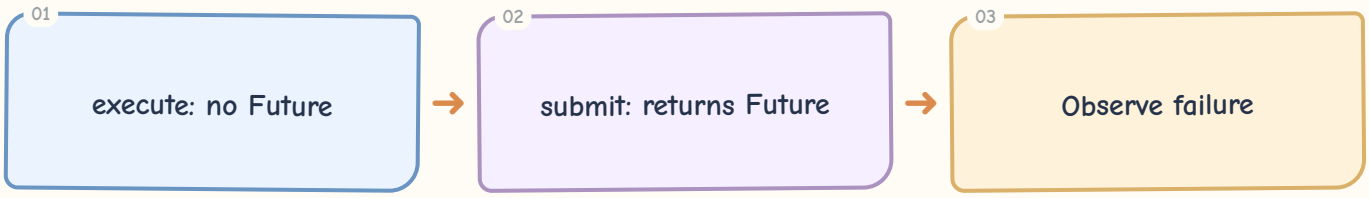
Q037. How do execute and submit differ?

THE 30-SECOND ANSWER

execute() runs a Runnable without returning a Future. submit() returns a Future, which can hold the task's result or failure; forgetting to inspect it can hide errors.

Why it matters: Unobserved Futures can make failing work look silent.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
pool.execute(() -> doWork());
Future<?> f = pool.submit(() -> doWork());
f.get(); // submitted exception is wrapped in ExecutionException
```

INTERVIEWER'S FOLLOW-UP

How would you report fire-and-forget work without losing failures?

Answer → Wrap tasks with logging/metrics and a clear failure callback, or retain Futures and observe every result; avoid silently discarding exceptions.

COMMON TRAP Merely submitting is not evidence that the task completed successfully.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Executors, Futures & lifecycle

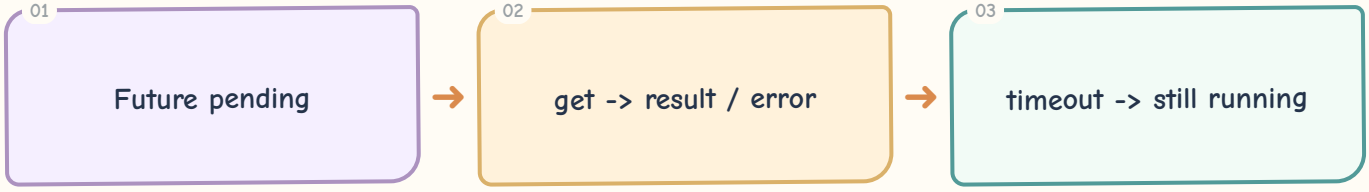
Q038. How do get, isDone and timeout differ on a Future?

THE 30-SECOND ANSWER

Future.get() waits for a result or reports failure or cancellation. isDone() means completion state, not success. A timed get can return timeout while work continues.

Why it matters: Completion-state queries and successful task outcomes answer different questions.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
if (future.isDone()) { /* may have failed or been cancelled */ }
Result r = future.get(200, MILLISECONDS);
// Handle completion, failure, cancellation and timeout separately.
```

INTERVIEWER'S FOLLOW-UP

What policy should follow timeout for a non-idempotent remote operation?

Answer → Use an idempotency key, query durable operation status, and retry only under a contract that rules out duplicate charges/writes.

COMMON TRAP Timeout does not automatically cancel the running task.

MY ONE-LINE RECAP _____

Executors, Futures & lifecycle

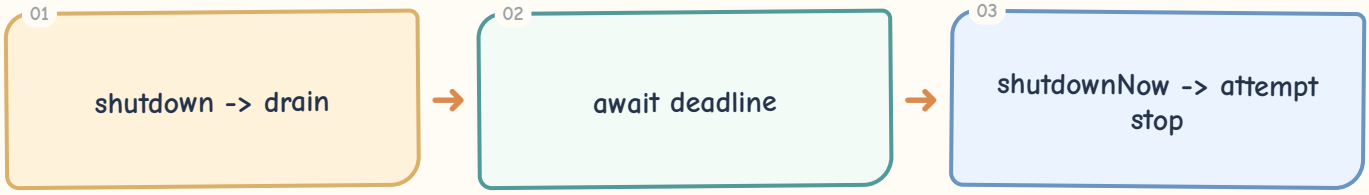
Q039. How do shutdown, shutdownNow and awaitTermination differ?

THE 30-SECOND ANSWER

shutdown() stops new submissions and drains accepted work. shutdownNow() attempts interruption and returns not-started tasks. awaitTermination() waits to see whether the pool really ends.

Why it matters: Orderly shutdown, interruption attempt and observed termination are separate steps.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
pool.shutdown();
if (!pool.awaitTermination(5, SECONDS)) pool.shutdownNow();
// Verify or report remaining work under a deadline policy.
```

INTERVIEWER'S FOLLOW-UP

How would you preserve the caller's interrupt status during shutdown?

Answer → Catch InterruptedException, call shutdownNow if policy allows, restore Thread.currentThread().interrupt(), and return or propagate without swallowing it.

! COMMON TRAP The returned queued tasks may still need cancellation or explicit business recovery.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Executors, Futures & lifecycle

Q040. What does ExecutorCompletionService solve?

THE 30-SECOND ANSWER

ExecutorCompletionService gives completed tasks back in completion order. It helps when tasks have different durations, but you still choose which result counts and cancel unneeded work.

Why it matters: The caller can consume available completions promptly.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
submit fast and slow independent tasks
Future<Result> finished = completion.take();
Result r = finished.get();
```

INTERVIEWER'S FOLLOW-UP

How would you cancel remaining tasks after choosing an acceptable result?

Answer → Keep every submitted Future, cancel unfinished ones, and wait a bounded time for cleanup; handle any side effects that cannot be undone.

COMMON TRAP Completion order is not request ordering or guaranteed first-success behavior.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

PART III

COMPOSITION & MODERN JAVA

Reason about completion stages, context propagation and virtual threads.

> Input → Stage → Completion

QUESTIONS 041–060

Keep your explanations short; make your concurrency guarantees exact.

Thread pools, queues & overload

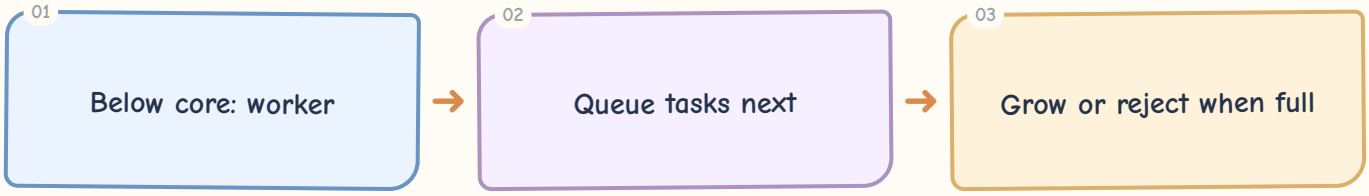
Q041. How does ThreadPoolExecutor decide between workers and queueing?

THE 30-SECOND ANSWER

ThreadPoolExecutor normally adds core workers first, queues next, grows toward max when queueing fails, then rejects. A large queue changes when extra workers are created.

Why it matters: Queue choice changes when maximum size becomes relevant.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
core=2, max=4, bounded queue capacity=10
First core workers -> queue -> additional workers -> rejection
Exact behavior also depends on concurrent completion.
```

INTERVIEWER'S FOLLOW-UP

Why might increasing maxPoolSize appear to have no effect?

Answer → With an unbounded queue, work is usually queued after core workers are occupied, so no queue rejection occurs to trigger growth toward maxPoolSize.

! COMMON TRAP An unbounded queue generally prevents growth beyond core size through this path.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread pools, queues & overload

Q042. What risks do common executor factory methods hide?

THE 30-SECOND ANSWER

Convenient executor factories hide resource choices. A fixed pool can still queue unlimited work; a cached pool can create many threads. Set explicit worker, queue and overload limits.

Why it matters: Convenient factories encode significant capacity policy.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
newFixedThreadPool(8): worker limit, not backlog limit
newCachedThreadPool(): flexible workers, not a strict cap
Inspect queue, workers and rejection together.
```

INTERVIEWER'S FOLLOW-UP

How would you bound both active and queued tasks?

Answer → Build ThreadPoolExecutor with a fixed maximum worker count, a bounded queue and explicit rejection. Also bound upstream admission where required.

⚠ COMMON TRAP A fixed number of workers does not make total accepted work bounded.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread pools, queues & overload

Q043. How do AbortPolicy and CallerRunsPolicy change overload behavior?

THE 30-SECOND ANSWER

AbortPolicy throws when a task is rejected. CallerRunsPolicy may run the task on the submitting thread, slowing it down; this can be dangerous on request or event-loop threads.

Why it matters: Rejection behavior affects latency, execution context and lifecycle.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE *original illustration*

```
Abort: caller observes rejection
CallerRuns: caller may execute the entire task
Shutdown: CallerRunsPolicy discards rather than runs
```

INTERVIEWER'S FOLLOW-UP

What if submission occurs while holding a lock the task needs?

Answer → CallerRunsPolicy may execute the task immediately while the submitter holds the lock, causing long stalls or self-deadlock if the task needs it.

⚠ COMMON TRAP Caller-runs is not universally safe backpressure and is not a durability guarantee.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread pools, queues & overload

Q044. How should you size a pool?

THE 30-SECOND ANSWER

Choose pool size using CPU limits, task wait time, downstream capacity, queue age and target latency. Measure under realistic load; no thread-count formula works for every service.

Why it matters: A sizing argument should name the actual bottleneck and limits.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Measure: useful throughput, p95/p99, CPU, queue age
Check: DB/HTTP capacity, memory, CPU quota
Change one policy and compare equivalent load.
```

INTERVIEWER'S FOLLOW-UP

When would a larger pool increase errors rather than throughput?

Answer → When more threads exceed DB/HTTP connection budgets or rate limits: dependency timeouts and rejected calls increase while successful throughput stagnates.

COMMON TRAP Host core count may not reflect a container's available CPU budget.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Thread pools, queues & overload

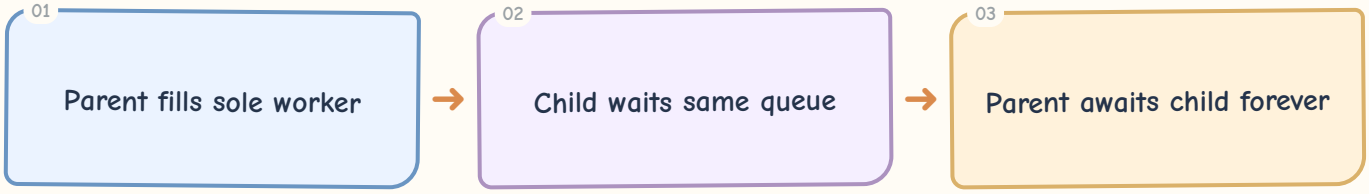
Q045. What is thread-pool starvation deadlock?

THE 30-SECOND ANSWER

Starvation deadlock happens when all pool workers wait for child tasks that are queued to that same full pool. Avoid blocking within the pool on work that needs that pool.

Why it matters: No monitor cycle is needed to produce this progress failure.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
Pool has one worker.
Parent runs -> submits child to same pool -> child.get()
Child waits in queue; parent occupies only worker.
```

INTERVIEWER'S FOLLOW-UP

How would you restructure the parent and child calculation?

Answer → Express child work through nonblocking stages (thenCompose/thenCombine), execute it directly if safe, or use separate appropriately sized execution resources.

COMMON TRAP Lock deadlock detection may not report a Future/queue dependency cycle.

Producer-consumer & coordination

Q046. How would you implement bounded producer-consumer handoff?

THE 30-SECOND ANSWER

Use a bounded BlockingQueue to hand finished items from producers to consumers. Decide what happens when it fills, and how to signal workers to stop without losing accepted tasks.

Why it matters: Queue capacity bounds queued entries but not all external producers or active work.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
BlockingQueue<Job> queue = new ArrayBlockingQueue<>(100);
boolean accepted = queue.offer(job, 100, MILLISECONDS);
Consumer: Job next = queue.take();
```

INTERVIEWER'S FOLLOW-UP

How would you stop several consumers safely?

Answer → After intake closes, enqueue one terminal marker per consumer or use cancellation/interrupt with a shared stop state. Drain accepted work before exits.

! COMMON TRAP A full-queue timeout needs explicit caller behavior; it is not silent success.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Producer-consumer & coordination

Q047. How do CountDownLatch and CyclicBarrier differ?

THE 30-SECOND ANSWER

CountDownLatch is a one-time countdown before work continues. CyclicBarrier brings a fixed set of workers together at repeated phase boundaries.

Why it matters: A one-shot completion gate and repeated rendezvous solve different problems.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Latch: N workers complete -> one observer proceeds
Barrier: N parties meet -> next shared phase
Choose by lifecycle, not by similar await method names.
```

INTERVIEWER'S FOLLOW-UP

Why should worker failure still signal a latch in finally?

Answer → Put countDown in finally so an exception does not leave the observer blocked forever; report task failure separately from completion.

COMMON TRAP A broken barrier is not ordinary successful phase completion.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Producer-consumer & coordination

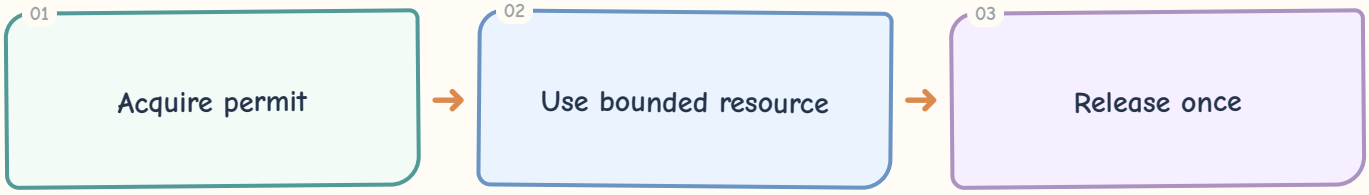
Q048. How does Semaphore control concurrency?

THE 30-SECOND ANSWER

Semaphore holds a limited number of permits. Acquire before using a limited resource, and release exactly once after the resource's work finishes.

Why it matters: The protected lifetime defines what the concurrency limit actually measures.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
permits.acquire();
try { callDependency(); }
finally { permits.release(); }
```

INTERVIEWER'S FOLLOW-UP

How would cancellation before queued execution affect permit ownership?

Answer → If admission owns permits before task execution, cancellation of a queued task must release its permit exactly once through a coordinated completion/cancel path.

⚠ COMMON TRAP Releasing a permit after failed acquisition inflates the budget.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Producer-consumer & coordination

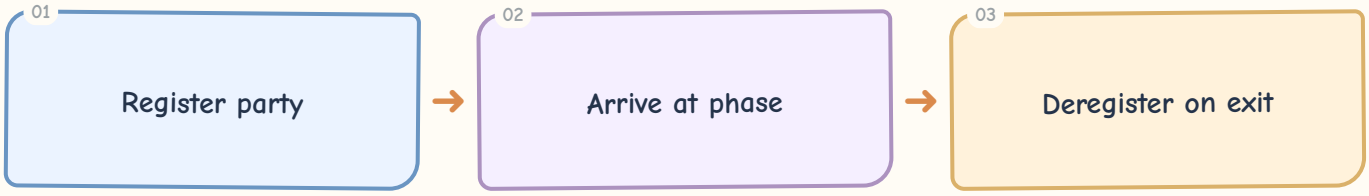
Q049. When is Phaser useful?

THE 30-SECOND ANSWER

Phaser helps when the set of workers can join or leave across repeated phases. Participants must register and always arrive or deregister when done.

Why it matters: Participant lifecycle is explicit rather than fixed at construction.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
phaser.register();
try { doPhaseWork(); }
finally { phaser.arriveAndDeregister(); }
```

INTERVIEWER'S FOLLOW-UP

What should happen if a participant fails before doing work?

Answer → Arrive and deregister in finally if registered; record failure and cancel/terminate the phase if the workflow cannot safely continue.

COMMON TRAP Registration without a terminal arrival can block phase progress.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Producer-consumer & coordination

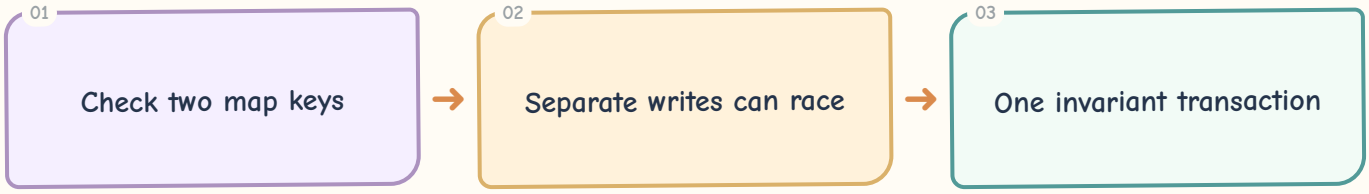
Q050. Are concurrent collections enough to make a workflow atomic?

THE 30-SECOND ANSWER

Concurrent collections protect their documented individual operations. A transfer involving two entries still needs a coordinated transaction, lock or single atomic state.

Why it matters: Operation safety and workflow safety are different levels.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
if (map.get(key) == null) map.put(key, value);
// Compare with putIfAbsent for this per-key decision.
// Mutable values still need their own policy.
```

INTERVIEWER'S FOLLOW-UP

How would you preserve a two-key transfer invariant?

Answer → Lock both keys in a stable order or use one transactional authority. Validate debit and credit together so partial changes are impossible.

⚠ COMMON TRAP A ConcurrentHashMap of ArrayLists does not make the lists safe to mutate concurrently.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

CompletableFuture composition

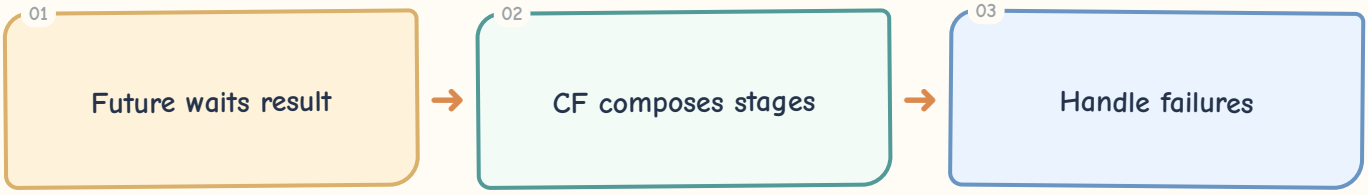
Q051. How do Future and CompletableFuture differ?

THE 30-SECOND ANSWER

Future mainly lets you wait for one outcome. CompletableFuture also lets you chain, combine and recover from stages. Code inside a stage may still block or fail.

Why it matters: The chain expresses dependencies through results rather than shared side channels.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
CompletableFuture.supplyAsync(this::load, ioPool)
.thenApply(this::validate)
.thenAccept(this::publish);
```

INTERVIEWER'S FOLLOW-UP

Which stages should use an explicit executor?

Answer → Use explicit executors for blocking network calls, heavy CPU work requiring isolation, or any stage whose capacity must be owned and limited.

! COMMON TRAP The word async does not describe the behavior of the work inside a callback.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

CompletableFuture composition

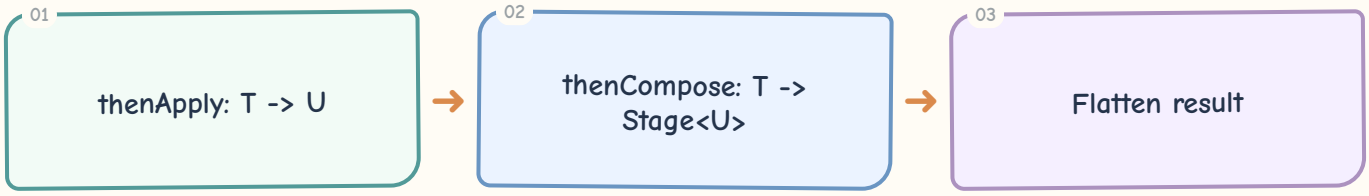
Q052. How do thenApply and thenCompose differ?

THE 30-SECOND ANSWER

thenApply changes one result into another result. thenCompose joins a stage that returns a further CompletionStage, avoiding nested futures.

Why it matters: Composition follows the dependent stage's completion.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
thenApply(id -> loadAsync(id)) // nested future result
thenCompose(id -> loadAsync(id)) // result future
// loadAsync returns CompletableFuture<Order>.
```

INTERVIEWER'S FOLLOW-UP

How would you combine independent loads instead?

Answer → Use thenCombine for two independent stages and apply the combination when both succeed; define the joint failure policy.

COMMON TRAP Returning a Future from apply does not cause it to be awaited as a flattened result.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

CompletableFuture composition

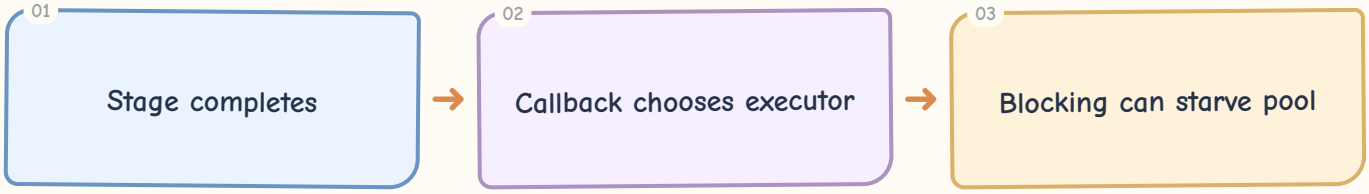
Q053. Where do async and non-async callbacks execute?

THE 30-SECOND ANSWER

Non-async callbacks may run in the thread completing the previous stage. Async variants use an executor, often the common pool unless one is supplied. Specify an executor for expensive or blocking work.

Why it matters: A callback can borrow a thread whose workload you did not expect.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
.thenApply(this::cheapTransform)
.thenApplyAsync(this::blockingLookup, dedicatedIoPool)
// Neither choice changes shared-state correctness requirements.
```

INTERVIEWER'S FOLLOW-UP

Why can blocking a shared common pool harm unrelated work?

Answer → The common pool is shared. Blocking its workers can delay unrelated async work and create starvation under load.

COMMON TRAP Non-async does not mean main thread, and async does not promise a new dedicated thread.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

CompletableFuture composition

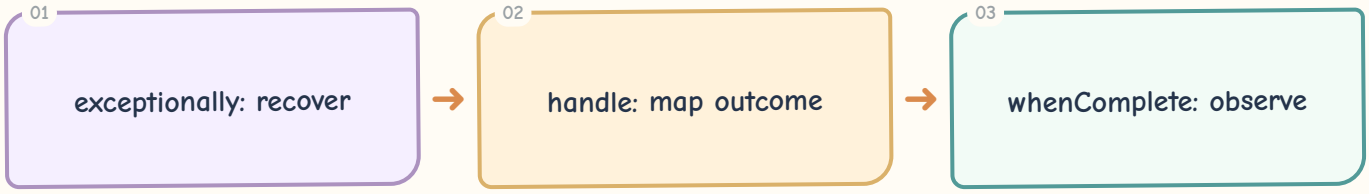
Q054. How do exceptionally, handle and whenComplete differ?

THE 30-SECOND ANSWER

exceptionally recovers from a failure, handle processes success or failure and supplies a result, and whenComplete observes an outcome without normally changing it. A callback can also fail.

Why it matters: Recovery, transformation and observation are different intentions.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
future.exceptionally(error -> fallback());
future.handle((value, error) -> resolve(value, error));
future.whenComplete((value, error) -> record(value, error));
```

INTERVIEWER'S FOLLOW-UP

What if the monitoring callback itself throws?

Answer → Its own exception can turn a previously successful stage into a failed dependent stage; log defensively and test error-monitoring paths.

⚠ COMMON TRAP A fallback must be semantically valid; replacing every failure with empty success can corrupt behavior.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

CompletableFuture composition

Q055. Do allOf, anyOf or orTimeout stop unnecessary work?

THE 30-SECOND ANSWER

allOf waits for all, anyOf finishes with the first completion (even failure), and orTimeout reports timeout. None automatically stops other tasks; that is your cleanup job.

Why it matters: Aggregation and timeout are outcome coordination, not universal resource cancellation.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
CompletableFuture.allOf(a, b).join();
// Collect individual results and handle failures explicitly.
// Timeout/cancellation needs a dependency-specific stop policy.
```

INTERVIEWER'S FOLLOW-UP

How would you implement first acceptable result with cleanup of losers?

Answer → Keep references to each competitor, select the first acceptable successful result, cancel losers and verify their resource cleanup under a deadline.

! COMMON TRAP CompletableFuture.cancel does not use interruption to control its computation as FutureTask may.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, fork/join & streams

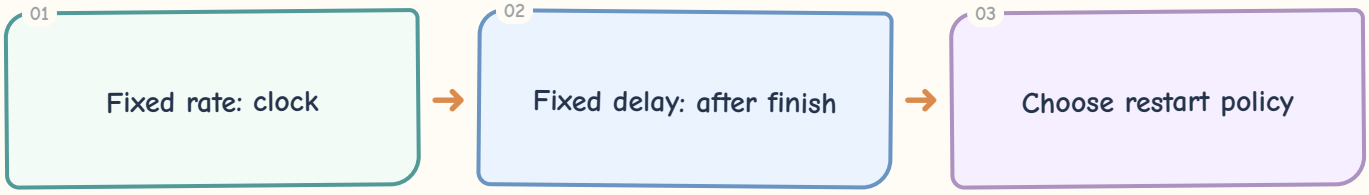
Q056. How do fixed-rate and fixed-delay scheduling differ?

THE 30-SECOND ANSWER

Fixed-rate scheduling targets a regular timetable; fixed-delay waits for one run to finish and then delays. Choose based on how missed or slow work should behave.

Why it matters: Neither option guarantees real-time execution or an application SLA.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Fixed rate: intended starts at 0, 10, 20...
Fixed delay: complete -> wait 10 -> next start
Long run duration changes effective timing.
```

INTERVIEWER'S FOLLOW-UP

Which policy suits maintenance that should rest between runs?

Answer → Fixed-delay is often better: it waits for the previous maintenance run to finish, then rests before the next attempt.

COMMON TRAP A slow fixed-rate task is not automatically executed concurrently with itself to catch up.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, fork/join & streams

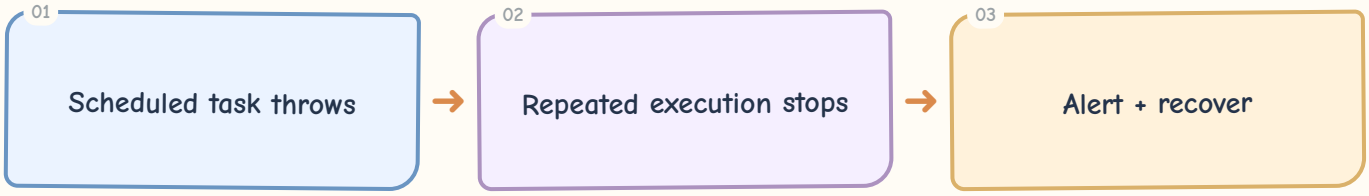
Q057. What happens if a periodic task throws an exception?

THE 30-SECOND ANSWER

An exception escaping a scheduled periodic task normally stops its future repetitions. Capture and report failures, then define whether the next run should still happen.

Why it matters: A started scheduler can be alive while one intended periodic job has stopped.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
scheduler.scheduleWithFixedDelay(() -> refresh(), 0, 10, SECONDS);
// A thrown unchecked exception stops this periodic sequence.
Observe the returned ScheduledFuture and health signals.
```

INTERVIEWER'S FOLLOW-UP

How would you test and alert on missed successful refreshes?

Answer → Track last successful refresh, freshness age and failed-run count; make the test throw once and verify no later run occurs without a recovery policy.

⚠ COMMON TRAP Monitoring worker existence alone cannot prove job execution continues.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, fork/join & streams

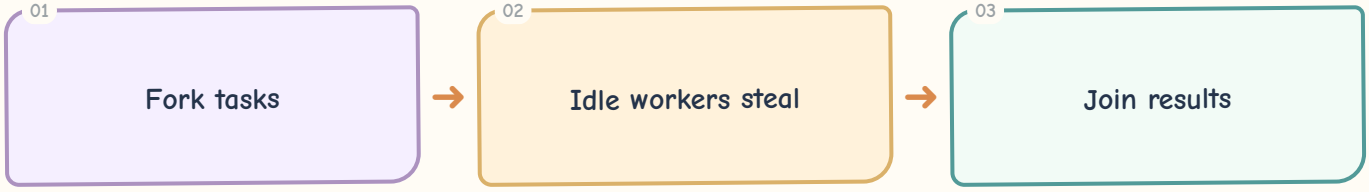
Q058. What does fork/join work stealing provide?

THE 30-SECOND ANSWER

Fork/join splits suitable CPU tasks and lets idle workers steal available work. A task blocked on slow I/O can waste scarce compute workers.

Why it matters: The algorithm's work size and dependency structure matter.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Split range -> compute left and right -> combine
Use a threshold to avoid one task per tiny operation.
Measure parallel overhead versus useful computation.
```

INTERVIEWER'S FOLLOW-UP

What is ManagedBlocker intended to help with?

Answer → ManagedBlocker helps ForkJoinPool compensate for supported blocking sections by managing parallelism, but it does not make arbitrary I/O free.

⚠ COMMON TRAP Blocking external I/O in a computation pool can reduce useful CPU parallelism.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, fork/join & streams

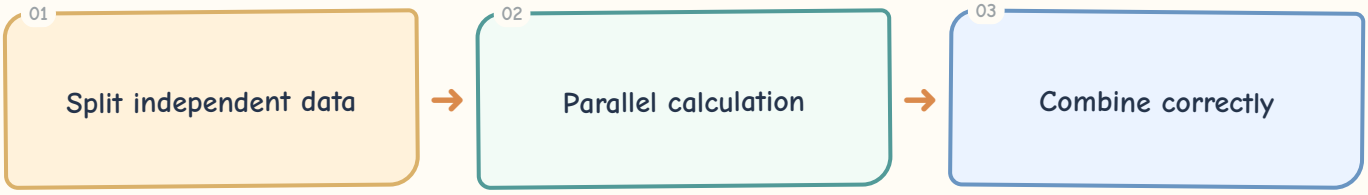
Q059. When are parallel streams appropriate?

THE 30-SECOND ANSWER

Parallel streams help with large independent CPU operations that are safe to split. They may hurt small, blocking or order-sensitive work.

Why it matters: A reduction is easier to reason about than concurrent mutation of an external list.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
long sum = values.parallelStream()
.mapToLong(this::costlyPureFunction).sum();
// Compare with sequential under the same workload.
```

INTERVIEWER'S FOLLOW-UP

Why does encounter order matter for some stream operations?

Answer → Stateful or ordered operations such as findFirst and ordered collection may reduce parallel benefit; dropping order is safe only if the result contract allows it.

COMMON TRAP Parallel is not always faster and does not make a non-thread-safe accumulator safe.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Scheduling, fork/join & streams

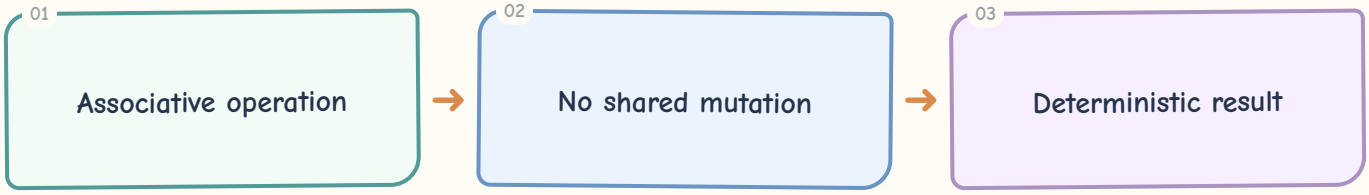
Q060. Why must parallel reductions use suitable functions?

THE 30-SECOND ANSWER

Parallel reduction only works correctly when its functions follow the needed associativity and non-interference rules. Shared mutable accumulators can corrupt results.

Why it matters: The result contract matters before measuring speed.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Bad reduction: subtraction as a generic associative combine
Safer example: long integer sum within the intended range
Floating-point totals: define reproducibility/tolerance requirements.
```

INTERVIEWER'S FOLLOW-UP

How would you handle deterministic monetary aggregation?

Answer → Use exact decimal arithmetic such as BigDecimal, associative combination under a defined rounding policy and no shared mutable collector.

⚠ COMMON TRAP A parallel run that happens to match one sequential result is not an algebraic proof.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

PART IV

CODING & SENIOR SCENARIOS

Prove the guarantee, test failures and defend the production tradeoff.

> Invariant → Failure → Evidence

QUESTIONS 061–080

Keep your explanations short; make your concurrency guarantees exact.

ThreadLocal & request context

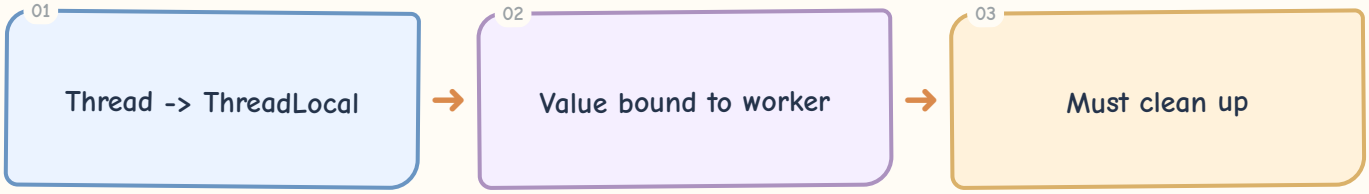
Q061. What does ThreadLocal provide?

THE 30-SECOND ANSWER

ThreadLocal stores a separate value per thread for one ThreadLocal key. It does not make the value immutable or automatically move it to another task.

Why it matters: Cleanup prevents a later request on the same worker from seeing stale binding state.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
context.set(requestContext);
try { process(); }
finally { context.remove(); }
```

INTERVIEWER'S FOLLOW-UP

What if requestContext contains a shared mutable list?

Answer → The per-thread binding may still point to the same shared ArrayList. Use immutable values or separate state ownership; ThreadLocal does not copy objects.

COMMON TRAP Thread-local reference and thread-confined object graph are different properties.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

ThreadLocal & request context

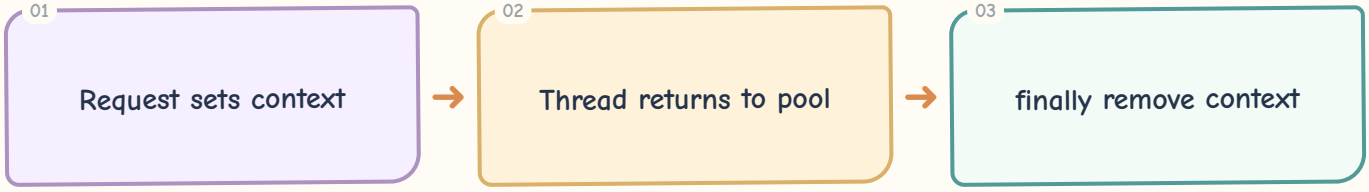
Q062. Why can ThreadLocal retain memory in a pool?

THE 30-SECOND ANSWER

A pool reuses threads, so ThreadLocal values can survive one request and leak into another. Remove them in finally when their scope ends.

Why it matters: The worker's lifetime amplifies a forgotten request scope.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Pool worker lives for hours
Request stores a large context value
Missing remove -> value may outlive request
```

INTERVIEWER'S FOLLOW-UP

How would you test cleanup on an exceptional request path?

Answer → Run a task that sets context and throws; then run another task on the same single pool worker and assert its context is absent.

⚠ COMMON TRAP Completing the request does not automatically terminate or clean every worker binding.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

ThreadLocal & request context

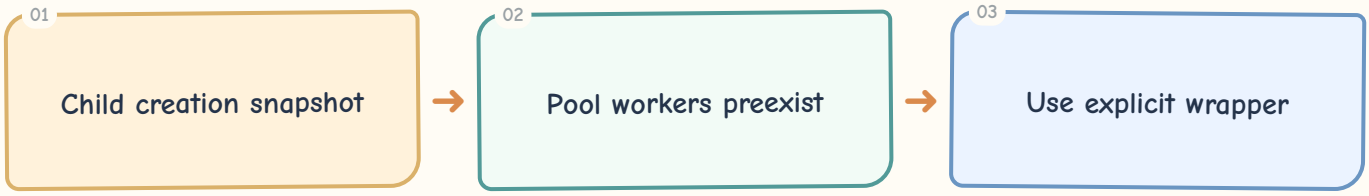
Q063. Does InheritableThreadLocal propagate every submitted request?

THE 30-SECOND ANSWER

InheritableThreadLocal copies context at child-thread creation; pooled threads were created earlier. It cannot reliably carry a new request's values through existing workers.

Why it matters: Creation inheritance and per-task propagation have different lifecycles.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Parent context at worker creation: A
Later submit request with context B
Existing worker does not automatically inherit B.
```

INTERVIEWER'S FOLLOW-UP

How would a context wrapper restore a worker's previous binding?

Answer → Before running, save the worker's old value, install the captured immutable request context, and restore/remove the previous value in finally.

⚠ COMMON TRAP Sharing inherited mutable values can still create data races.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

ThreadLocal & request context

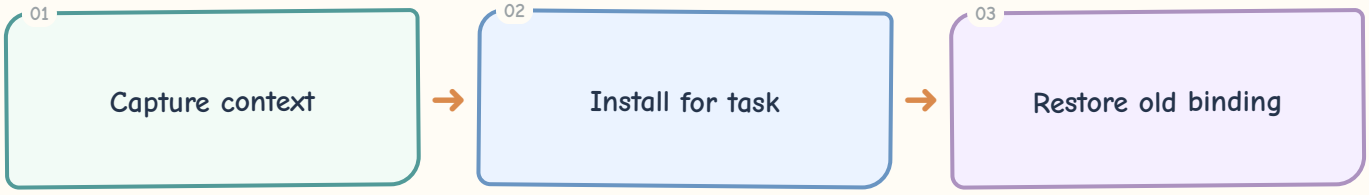
Q064. How should request context cross an asynchronous boundary?

THE 30-SECOND ANSWER

Carry request context as explicit immutable data or through a well-defined context wrapper. Save previous worker state and restore it afterward.

Why it matters: The wrapper's contract defines what moves and when it ends.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Context captured = currentContext();
pool.submit(() -> runWithContext(captured, task));
// Wrapper restores previous context after task, including failure.
```

INTERVIEWER'S FOLLOW-UP

Which context is deliberately unsafe to propagate to concurrent work?

Answer → Do not propagate open transactions, mutable request-scoped entity managers or thread-affine database connections to parallel tasks.

! COMMON TRAP Removing rather than restoring can break nested scopes that already had context.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

ThreadLocal & request context

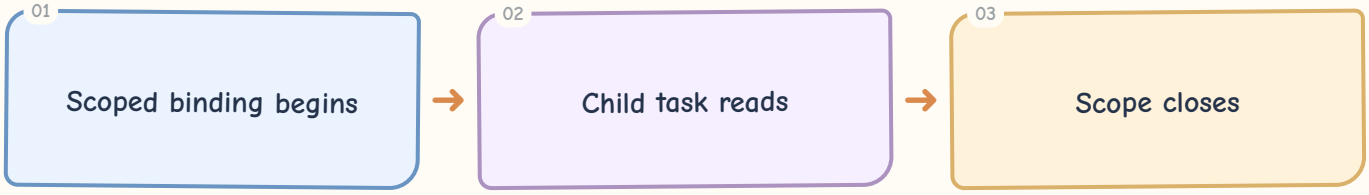
Q065. What are Scoped Values, and are they a ThreadLocal replacement for everything?

THE 30-SECOND ANSWER

Scoped Values provide a bounded, read-oriented context-sharing model on supporting newer Java runtimes. They are not a general replacement for mutable ThreadLocal state.

Why it matters: A scope-oriented lifetime reduces some context-cleanup complexity.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE *original illustration*

```
Java 25 conceptual flow:
bind context -> run bounded scope -> binding ends
Review the scoped API and task relationship before propagation.
```

INTERVIEWER'S FOLLOW-UP

Which mutable per-thread use case might still require another mechanism?

Answer → Mutable per-thread scratch buffers that require updates throughout a task may still need explicit local state or ThreadLocal with cleanup.

! COMMON TRAP Java 17/21 source cannot use the finalized Java 25 API unchanged.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Virtual threads & modern concurrency

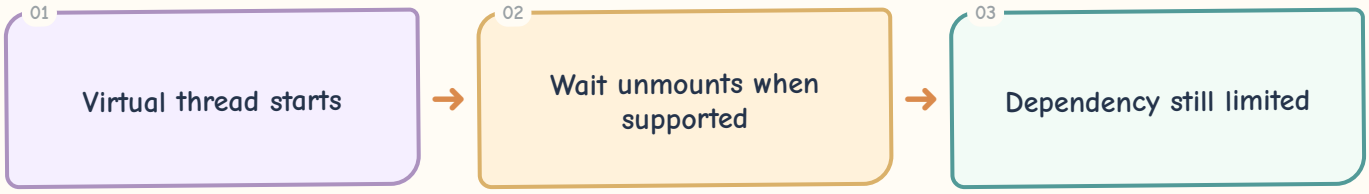
Q066. How do virtual threads differ from platform threads?

THE 30-SECOND ANSWER

Virtual threads are lightweight Java-managed threads built for many waiting tasks. They reduce thread-per-task overhead, but CPU, RAM, databases and remote APIs still have finite capacity.

Why it matters: A familiar blocking style can support more concurrent waiting work.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Many blocking tasks -> virtual threads -> carrier execution
Database access -> separate finite connection budget
Final feature: Java 21
```

INTERVIEWER'S FOLLOW-UP

How would you control offered load when waiting becomes cheaper?

Answer → Use admission limits, rate control, semaphores and bounded connection pools at the expensive dependency, not a pool of virtual threads.

⚠ COMMON TRAP They are not a speed multiplier for a long CPU-bound computation.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Virtual threads & modern concurrency

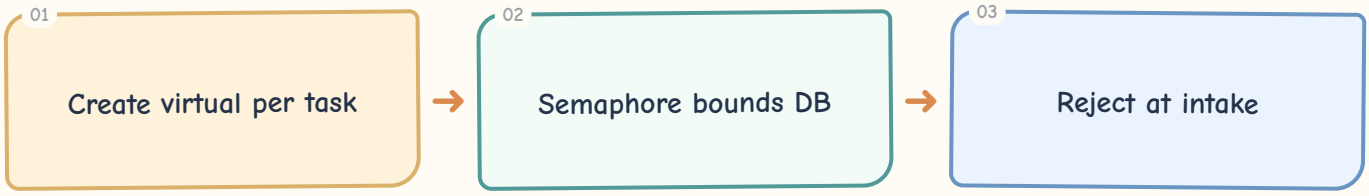
Q067. Should you pool virtual threads?

THE 30-SECOND ANSWER

Normally create virtual threads per task instead of pooling them. Bound requests or database calls with semaphores, queues and other admission controls.

Why it matters: Thread creation policy and resource concurrency become separate decisions.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Java 21+: Executors.newVirtualThreadPerTaskExecutor()
Limit dependency calls with an explicit semaphore/admission policy.
Own task completion and executor lifecycle.
```

INTERVIEWER'S FOLLOW-UP

Where should overload be rejected before too much work accumulates?

Answer → Reject or queue at the API/intake boundary before creating enormous numbers of tasks and payloads; keep a measured in-flight cap.

COMMON TRAP A per-task executor alone does not bound retained in-flight requests.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Virtual threads & modern concurrency

Q068. What is virtual-thread pinning, and what changed after Java 21?

THE 30-SECOND ANSWER

Pinning is when a virtual thread cannot unmount from its carrier during some blocking work. Its causes and behavior have changed across Java releases, so profile the deployed version.

Why it matters: Reduced pinning does not remove contention or resource bottlenecks.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE *original illustration*

```
Java 21: review blocking inside synchronized sections
Java 24+: monitor-related pinning change
All versions: locks can still serialize application work
```

INTERVIEWER'S FOLLOW-UP

Which evidence would justify a lock redesign on your deployed version?

Answer → Gather Java-version-specific JFR pinning events, thread dumps and throughput during blocking sections. Redesign locks only if the deployed runtime shows cost.

⚠ COMMON TRAP 'Replace every synchronized with ReentrantLock' is not a universal modern-JDK rule.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Virtual threads & modern concurrency

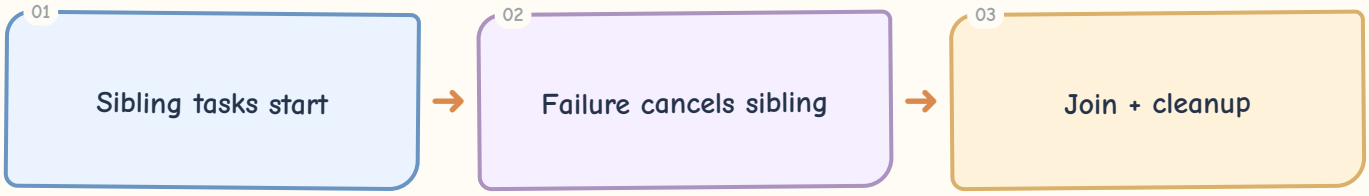
Q069. What is structured concurrency, and what is its Java 25 status?

THE 30-SECOND ANSWER

Structured concurrency groups related tasks under one lifecycle so failure, cancellation and joining can be managed together. Check the exact Java 25 preview API before using it.

Why it matters: The model makes task relationships and failure cleanup easier to express.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
Request scope -> fork related tasks -> join policy -> scope closes
Children should not silently outlive their intended parent unit.
Java 25: fifth preview; not executed in this guide.
```

INTERVIEWER'S FOLLOW-UP

What failure policy should cancel a sibling dependency lookup?

Answer → If one lookup fails and the response cannot be built, cancel its sibling, await cleanup, and propagate the original failure with a bounded deadline.

! COMMON TRAP Do not copy a Java 21 preview signature and assume it matches Java 25.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Virtual threads & modern concurrency

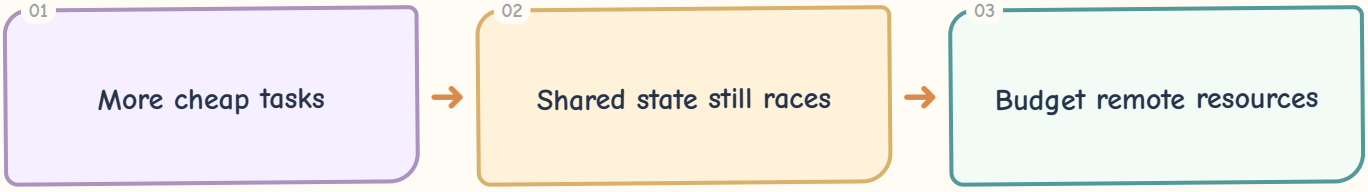
Q070. Does virtual-thread adoption eliminate deadlocks or ThreadLocal costs?

THE 30-SECOND ANSWER

Virtual threads do not remove data races, deadlocks or bad context ownership. Large numbers of them can still consume memory and saturate dependencies.

Why it matters: Changing execution economics can expose limits that were previously hidden by small pools.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Many virtual tasks x large per-thread cache -> substantial memory
Same shared lock order -> same potential dependency cycle
Measure real workload and diagnostic coverage.
```

INTERVIEWER'S FOLLOW-UP

Which pool-oriented optimization would you remove before migration?

Answer → Remove the assumption that one pooled thread represents one request for its whole lifetime. Review stale ThreadLocals and connection pools separately.

! COMMON TRAP Cheap waiting is not cheap unlimited application state.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Coding challenges & reproducible tests

Q071. How do you print odd and even numbers in order safely?

THE 30-SECOND ANSWER

Use one synchronized next-number variable and a while-loop condition for odd and even workers. Notify after updating, and allow both workers to stop if one fails.

Why it matters: The full source in the labs tests exact ordering without actor logging.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
while (next <= limit && next % 2 != myParity) lock.wait();
if (next > limit) { lock.notifyAll(); return; }
output(next++); lock.notifyAll(); // under the same monitor
```

INTERVIEWER'S FOLLOW-UP

What should happen if one participant fails halfway through?

Answer → Set shared cancelled state, notifyAll/interrupt the other worker, then join both and report partial output or failure without hanging.

! COMMON TRAP Sleeping one worker longer does not establish alternating order.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Coding challenges & reproducible tests

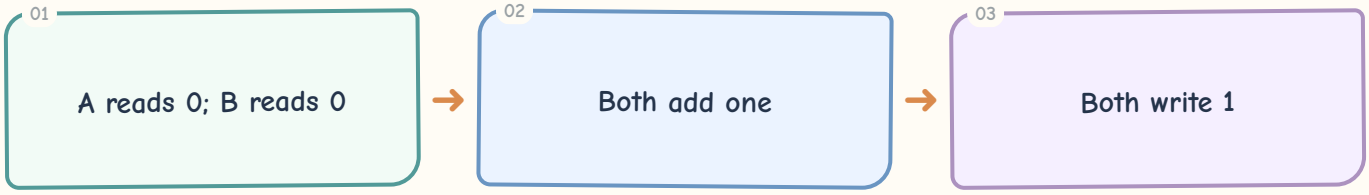
Q072. How would you demonstrate a lost update reliably?

THE 30-SECOND ANSWER

Make two threads read the same count before either writes. Then both write the same incremented result, showing that a volatile increment can lose an update.

Why it matters: Two logical increments can produce one stored increment.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
A reads count=0; B reads count=0
Barrier lets both continue
A writes 1; B writes 1 -> final 1
```

INTERVIEWER'S FOLLOW-UP

Which atomic operation removes this read/write gap?

Answer → AtomicInteger.incrementAndGet() performs the whole increment as one atomic update, avoiding the separate read/write window.

! COMMON TRAP Coordination changes scheduling; report exactly what it controls rather than claiming an unconstrained stress result.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Coding challenges & reproducible tests

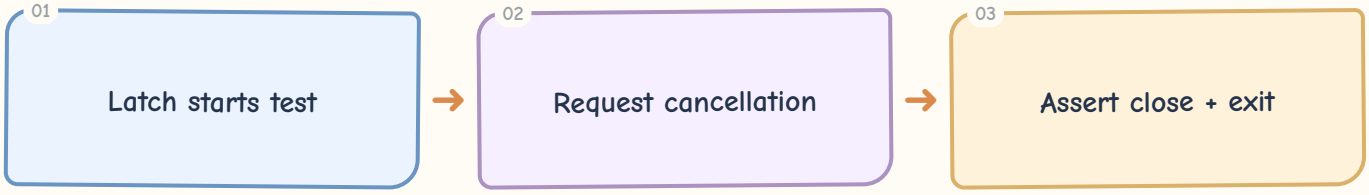
Q073. How would you test cancellation without depending on sleeps?

THE 30-SECOND ANSWER

Use latches, controlled tasks and a completion deadline to test cancellation. Assert that interruption was received, cleanup completed and the task truly ended.

Why it matters: The test controls relevant milestones and checks the promised lifecycle.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Worker signals entered -> waits interruptibly
Caller waits entered -> interrupts -> waits cleaned up
Caller verifies actual termination and cleanup
```

INTERVIEWER'S FOLLOW-UP

What additional assertion would detect a leaked resource?

Answer → Assert that the resource's close/release hook ran exactly once and the worker terminated, not merely that its Future shows cancelled.

COMMON TRAP A cancelled Future alone is not proof that finally completed.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Coding challenges & reproducible tests

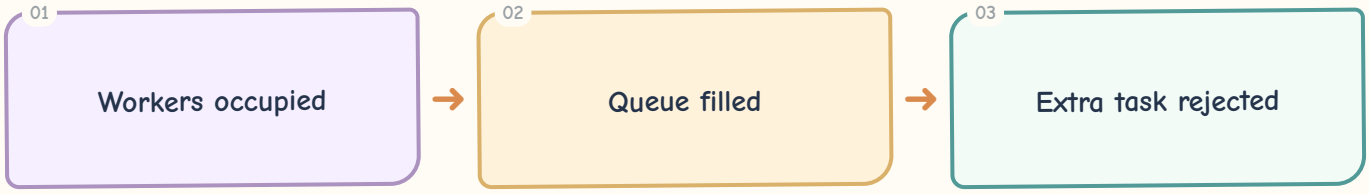
Q074. How do you test bounded pool rejection deterministically?

THE 30-SECOND ANSWER

Occupy the pool workers, fill the bounded queue with held tasks, then submit one extra task. That extra task should exercise the selected rejection policy deterministically.

Why it matters: Acceptance capacity is exercised under controlled occupancy.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
Pool: 1 worker, queue capacity 1, AbortPolicy
Task 1 occupies worker; task 2 fills queue
Task 3 -> RejectedExecutionException
```

INTERVIEWER'S FOLLOW-UP

How would CallerRunsPolicy change this test?

Answer → CallerRunsPolicy executes the rejected task on the submitter while the pool is active. The test must assert that execution context and avoid deadlocking itself.

! COMMON TRAP Without a gate, fast tasks may complete before the third submission and hide the intended condition.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Coding challenges & reproducible tests

Q075. Why is a passing stress test not proof of thread safety?

THE 30-SECOND ANSWER

Passing a finite stress test shows only the interleavings observed. Correctness must be argued using shared state, atomicity and happens-before, with tests supporting the proof.

Why it matters: Use evidence to complement the contract, not to redefine it.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Reasoning: allowed outcomes and missing edges
Testing: observed outcomes and failures
Performance: useful throughput, latency and resource cost
```

INTERVIEWER'S FOLLOW-UP

Which coordination edges belong only to the test observer?

Answer → The test's latches and joins create observer ordering that might not exist in production. Review the actors' own synchronization separately.

⚠ COMMON TRAP Logging, sleep or a test latch can accidentally add ordering absent from the production code.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Senior production scenarios

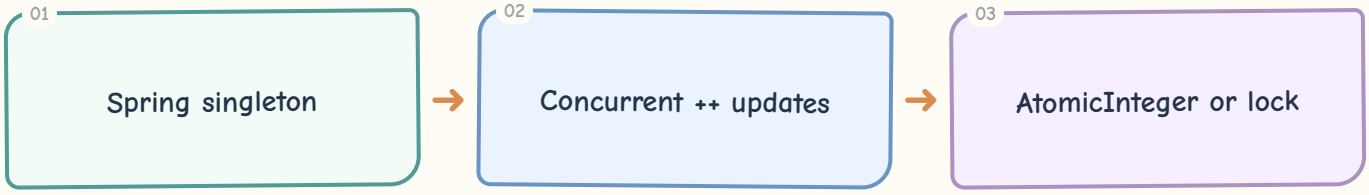
Q076. A Spring service's counter loses increments. What would you review?

THE 30-SECOND ANSWER

A Spring singleton is shared by many request threads. If it modifies a plain or volatile count with ++, updates can disappear; use a suitable atomic counter or lock.

Why it matters: Scope controls instance sharing, not business-operation synchronization.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE *original illustration*

```
@Service class Tracker { int count; void record() { count++; } }
Local exact counter: AtomicInteger/AtomicLong
Cluster-wide count: independent distributed design
```

INTERVIEWER'S FOLLOW-UP

When would a metric accumulator be sufficient?

Answer → LongAdder can be enough for high-throughput approximate live metrics when exact concurrent snapshots aren't needed. Use atomic/lock rules for strict limits.

! COMMON TRAP Fixing one JVM counter does not create a global exact total across replicas.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Senior production scenarios

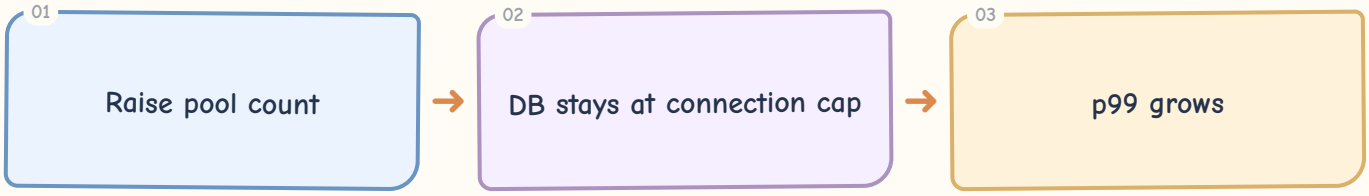
Q077. An API gets slower after the executor size increases. Why?

THE 30-SECOND ANSWER

More pool threads can pile up calls behind a fixed database pool, increasing wait time and timeouts. Compare queue age, DB active connections, CPU and successful throughput.

Why it matters: The executor change can shift pressure downstream rather than add capacity.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE *original illustration*

```
More workers -> more JDBC waiters -> longer request residence
Check acquisition time separately from SQL execution time.
Count replicas x per-process pool budgets.
```

INTERVIEWER'S FOLLOW-UP

What would disprove the database-capacity hypothesis?

Answer → If DB pool has spare connections, DB wait stays low, and successful throughput grows with threads while p99 remains controlled, DB capacity is less likely the cause.

COMMON TRAP Improving task-start latency while worsening end-to-end p99 is not a service improvement.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Senior production scenarios

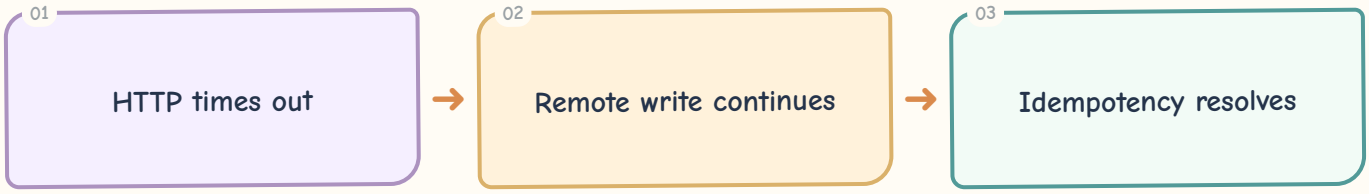
Q078. A timeout response returns, but downstream writes continue. What happened?

THE 30-SECOND ANSWER

Returning an HTTP timeout does not undo work already submitted or a remote write. Design cancellation, downstream deadlines and request idempotency independently.

Why it matters: Outcome semantics must survive the gap between caller timeout and remote completion.

DRAW IT TO REMEMBER ✦



JAVA / PSEUDOCODE EXAMPLE

original illustration

```
Client stops waiting at deadline
Task/remote server can still complete side effect
Represent unknown outcome; retry only under a correct policy
```

INTERVIEWER'S FOLLOW-UP

What key and persistence rule would support idempotent retry?

Answer → Use a stable request identifier with a unique durable record. Return the stored result on replay, including after a timeout with uncertain commit outcome.

! COMMON TRAP Blind retries can duplicate an operation even if local interruption was requested.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Senior production scenarios

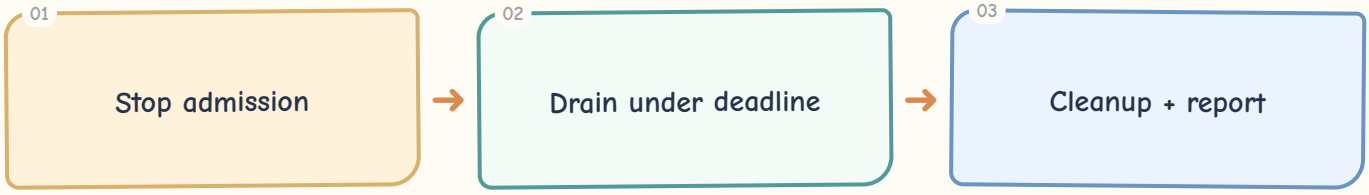
Q079. How would you shut down a background processing service?

THE 30-SECOND ANSWER

Stop accepting new jobs, drain accepted work under a deadline, request cancellation if necessary, clean up resources and report what actually completed.

Why it matters: Shutdown correctness includes business recovery, not only terminating worker threads.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE *original illustration*

```
Stop accepts -> drain/cancel accepted tasks -> cleanup
Deadline expires -> report/recover unfinished jobs
Durability belongs to the chosen queue/store policy
```

INTERVIEWER'S FOLLOW-UP

What if a worker blocks in a dependency during shutdown?

Answer → Set an I/O timeout, attempt supported interruption/connection close, wait a limited grace period and quarantine/report unfinished operations before shutdown.

! COMMON TRAP shutdownNow is a best-effort interruption request, not guaranteed drain or durable recovery.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Senior production scenarios

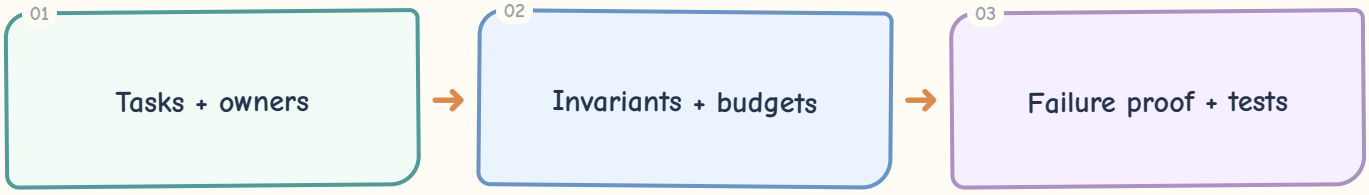
Q080. How do you review a threads-related design as a technical lead?

THE 30-SECOND ANSWER

As a lead, list tasks, owners, shared state and bottlenecks. Require explicit rules for synchronization, timeouts, overload, cancellation, shutdown, observability and tests.

Why it matters: The review connects Java semantics with customer-visible behavior and operations.

DRAW IT TO REMEMBER ✦



EXAMPLE FROM THE GUIDE

original illustration

```
Task -> owner -> executor -> resource budget
Shared state -> invariant -> synchronization policy
Failure -> cancellation -> cleanup -> observed completion
```

INTERVIEWER'S FOLLOW-UP

What documentation stops future code from bypassing the ownership policy?

Answer → Document ownership per executor and state field, invariants, lock order, cancellation/deadline policy, queue capacity and how to test these contracts.

! COMMON TRAP A sophisticated executor configuration cannot compensate for a missing state or cancellation contract.

One idea · One example · One follow-up · Java 17 baseline unless marked otherwise

MY ONE-LINE RECAP _____

Pick the right tool.

Need an exact shared count?	AtomicInteger.incrementAndGet or an appropriate lock.
Need to guard multiple fields?	One shared lock or one immutable snapshot updated atomically.
Need one-way state publication?	A correct volatile/release-acquire handoff.
Need to wait for a predicate?	Condition/Object.wait in a loop, with a bound if needed.
Need N workers to finish?	CountDownLatch or Futures, with failure handling.
Need a hard downstream concurrency cap?	Semaphore with acquire/release ownership.
Need a producer-consumer handoff?	Bounded BlockingQueue and clear shutdown protocol.
Need several independent results?	CompletableFuture thenCombine/allOf plus cleanup.
Need high-volume blocking tasks?	Virtual threads plus admission and dependency limits.
Need no duplicate remote side effect?	Durable idempotency state; threads alone cannot provide it.

Architect habit: Start from an invariant, not an API name. Say what must never happen, what can time out, and what proves completion.

Observe before tuning.

BLOCKED threads	Who owns the monitor? How long has it been held?
WAITING pool workers	Are they idle or waiting for work that cannot start?
Long queue age	Are arrivals exceeding successful completions?
Executor maxPoolSize ignored	Is an unbounded queue accepting all tasks first?
Frequent request timeouts	Do remote tasks keep running after responses finish?
Lost counter increments	Is the update split into read / compute / write?
Stale ThreadLocal values	Was remove() called in finally on reused workers?
Deadlock with no monitor cycle	Look for Future/queue dependency cycles.
Increasing threads but flat throughput	Check CPU, DB pool and upstream/downstream saturation.
Virtual-thread slowdowns	Measure pinning or blocking on the deployed Java release.

Record before/after: p95/p99, goodput, CPU, queue depth, in-flight tasks, DB pool wait, errors, cancellation completion.

10 ways to prove your understanding.

All ten source-guide Java 17 exercises were extracted, compiled, and run against an available Java 21 JDK using **--release 17**. These are bounded training checks, not production benchmarks or full concurrency proofs.

Build → **Run** → **Assert** → **Explain**

L01	Direct run, start and restart	LifecycleDemo
L02	Alternating output with a guarded predicate	OddEven
L03	Interruption, cleanup and actual termination	CancelDemo
L04	A deterministic overload boundary	PoolRejection
L05	Bounded handoff and termination marker	QueuePipeline
L06	Context removal after an exception	ContextCleanup
L07	Independent combine and dependent compose	CompletionChain
L08	An uncaught periodic failure stops recurrence	PeriodicFailure
L09	Exact updates and observed completion	AtomicTotals
L10	Controlled volatile lost update	LostUpdate

Always run the examples in a scratch directory. None of the exercises needs AWS, Spring Boot, Kafka or a running web service.

HANDS-ON · JAVA 17

L01. Direct run, start and restart

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

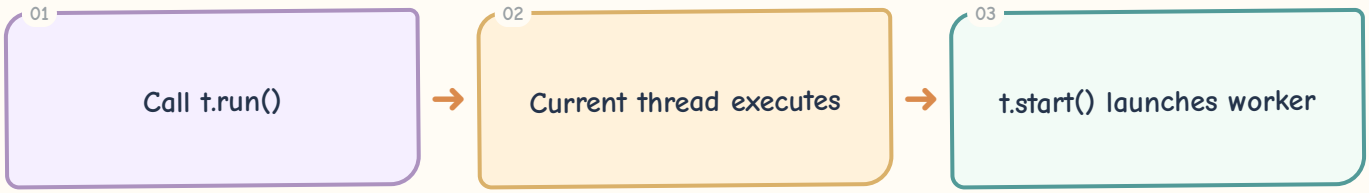
```
javac --release 17 *.java
java LifecycleDemo
```

EXPECTED CHECK

```
run=caller; start=worker; restart rejected
```

Explain the result

Explain why run() has no new thread, why start() does, and why a terminated Thread cannot be reused.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L02. Alternating output with a guarded predicate

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

```
javac --release 17 *.java
java OddEven
```

EXPECTED CHECK

ordered output: 1 through 20

Explain the result

Identify the shared next value, guarded while predicate, notifyAll and the cancellation exit.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L03. Interruption, cleanup and actual termination

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

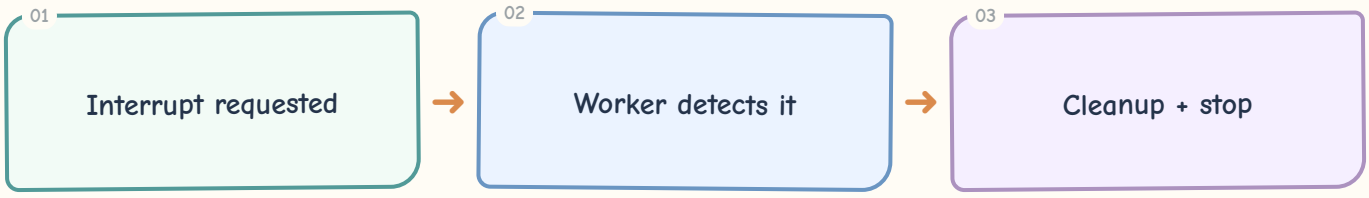
```
javac --release 17 *.java
java CancelDemo
```

EXPECTED CHECK

```
cancelled; cleanup completed; worker terminated
```

Explain the result

Check the difference between requesting cancellation, cleanup completion and an actual stopped worker.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L04. A deterministic overload boundary

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

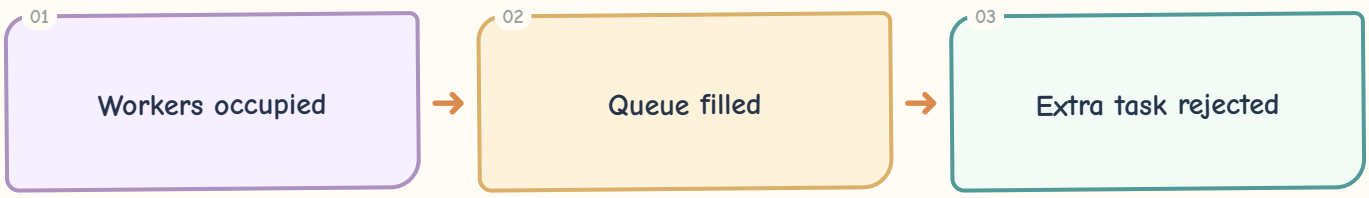
```
javac --release 17 *.java
java PoolRejection
```

EXPECTED CHECK

```
third task rejected; accepted tasks completed
```

Explain the result

Show why a full bounded queue plus busy workers forces a rejection instead of infinite backlog.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L05. Bounded handoff and termination marker

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

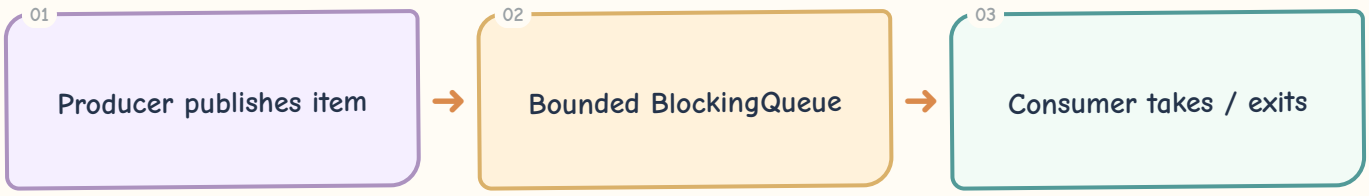
```
javac --release 17 *.java
java QueuePipeline
```

EXPECTED CHECK

```
100 items consumed; sum=5050
```

Explain the result

Explain producer ownership, the queue limit, the termination marker and consumer completion.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L06. Context removal after an exception

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

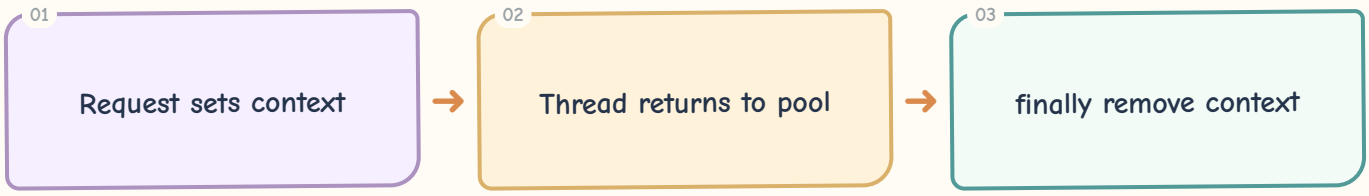
```
javac --release 17 *.java
java ContextCleanup
```

EXPECTED CHECK

`exceptional scope cleaned; next request has no context`

Explain the result

Prove ThreadLocal.remove runs in finally even when the business operation fails.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L07. Independent combine and dependent compose

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

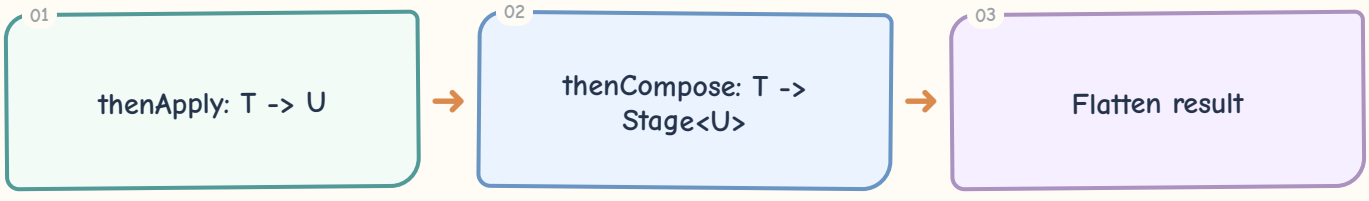
```
javac --release 17 *.java
java CompletionChain
```

EXPECTED CHECK

```
combine=42; compose=40; recovery=7
```

Explain the result

Compare thenCombine and thenCompose, and explain how exceptions are handled.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L08. An uncaught periodic failure stops recurrence

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

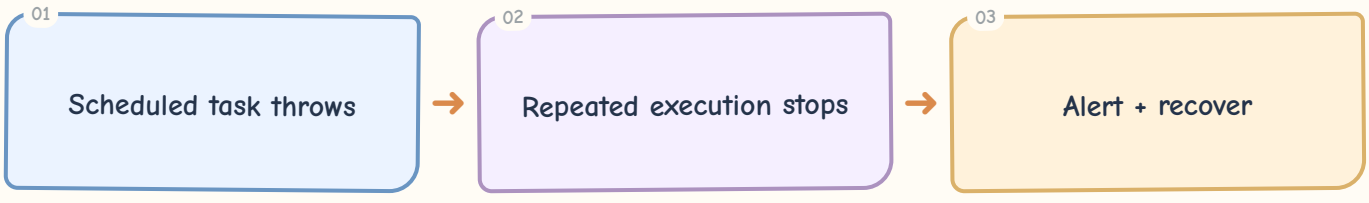
```
javac --release 17 *.java
java PeriodicFailure
```

EXPECTED CHECK

```
periodic task failed; executions=1
```

Explain the result

Explain why one failed periodic execution stops the next scheduled run.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L09. Exact updates and observed completion

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

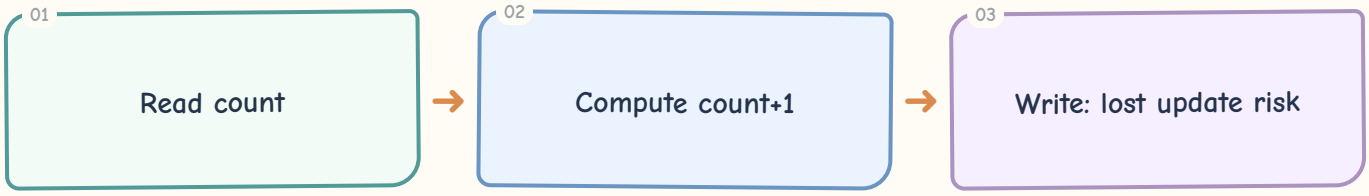
```
javac --release 17 *.java
java AtomicTotals
```

EXPECTED CHECK

```
atomic total=40000
```

Explain the result

Check that the atomic increments and observer join/get provide the required guarantees.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

HANDS-ON · JAVA 17

L10. Controlled volatile lost update

Use the matching, complete Java file in the source-code appendix. Run this lab in its own JVM process.

TERMINAL

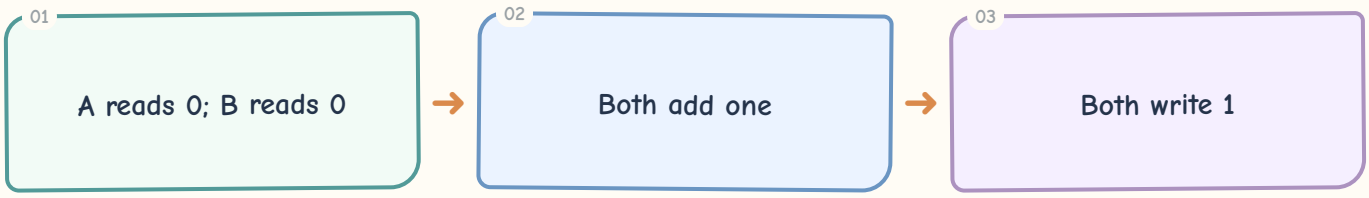
```
javac --release 17 *.java
java LostUpdate
```

EXPECTED CHECK

```
two increments, count=1
```

Explain the result

Trace both reads before both writes; explain why AtomicInteger removes the gap.



Lab boundary: A passing run shows this example's asserted behavior, not a universal guarantee for every schedule or deployment.

Read the whole program.

Interview snippets often hide setup or cleanup. The following listings reproduce the complete executable examples used in the original Java Threads source guide. Each program also appears as an individual .java file in the lab download.

Read → Predict → Run → Explain

Compile & run

```
cd java17
javac --release 17 *.java
java LifecycleDemo
java OddEven
```

A virtual-thread example for Java 21+ is placed after the Java 17 source listings.

LifecycleDemo.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.*;
public class LifecycleDemo {
  public static void main(String[] args) throws Exception {
    List<Thread> observed = new ArrayList<>();
    Thread caller = Thread.currentThread();
    Thread worker = new Thread(() -> observed.add(Thread.currentThread()));
    if (worker.getState() != Thread.State.NEW) throw new AssertionError();
    worker.run();
    worker.start(); worker.join();
    if (observed.size() != 2 || observed.get(0) != caller
        || observed.get(1) != worker) throw new AssertionError();
    if (worker.getState() != Thread.State.TERMINATED) throw new AssertionError();
    try { worker.start(); throw new AssertionError("restart succeeded"); }
    catch (IllegalThreadStateException expected) { }
    System.out.println("run=caller; start=worker; restart rejected");
  }
}
```

Run it: **java LifecycleDemo** · Source code is also in the ZIP bundle.

OddEven.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.*;
public class OddEven {
  final Object lock = new Object();
  final List<Integer> values = new ArrayList<>();
  int next = 1;
  boolean cancelled;
  void emit(int parity) {
    synchronized (lock) {
      try {
        while (!cancelled && next <= 20) {
          while (!cancelled && next <= 20 && next % 2 != parity)
            lock.wait();
          if (cancelled || next > 20) break;
          values.add(next++); lock.notifyAll();
        }
      } catch (InterruptedException e) {
        cancelled = true; Thread.currentThread().interrupt();
      } finally { lock.notifyAll(); }
    }
  }
  public static void main(String[] args) throws Exception {
    OddEven s = new OddEven();
    Thread odd = new Thread(() -> s.emit(1));
    Thread even = new Thread(() -> s.emit(0));
    odd.setDaemon(true); even.setDaemon(true);
    odd.start(); even.start(); odd.join(5000); even.join(5000);
    if (odd.isAlive() || even.isAlive()) {
      odd.interrupt(); even.interrupt(); throw new AssertionError("timeout");
    }
    if (s.values.size() != 20) throw new AssertionError();
    for (int i = 0; i < 20; i++)
      if (s.values.get(i) != i + 1) throw new AssertionError();
    System.out.println("ordered output: 1 through 20");
  }
}
```

Run it: **java OddEven** · Source code is also in the ZIP bundle.

CancelDemo.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.*;
public class CancelDemo {
  public static void main(String[] args) throws Exception {
    CountDownLatch entered = new CountDownLatch(1);
    CountDownLatch neverReleased = new CountDownLatch(1);
    CountDownLatch cleaned = new CountDownLatch(1);
    Thread worker = new Thread(() -> {
      try { entered.countDown(); neverReleased.await(); }
      catch (InterruptedException expected) {
        // Top-level worker consumes cancellation and exits.
      } finally { cleaned.countDown(); }
    });
    worker.setDaemon(true); worker.start();
    try {
      if (!entered.await(5, TimeUnit.SECONDS)) throw new AssertionError();
      worker.interrupt();
      if (!cleaned.await(5, TimeUnit.SECONDS)) throw new AssertionError();
      worker.join(5000);
      if (worker.isAlive()) throw new AssertionError();
      System.out.println("cancelled; cleanup completed; worker terminated");
    } finally { worker.interrupt(); }
  }
}
```

Run it: **java CancelDemo** · Source code is also in the ZIP bundle.

PoolRejection.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.*;
public class PoolRejection {
  public static void main(String[] args) throws Exception {
    CountDownLatch entered = new CountDownLatch(1);
    CountDownLatch gate = new CountDownLatch(1);
    ThreadPoolExecutor pool = new ThreadPoolExecutor(1, 1, 0,
        TimeUnit.SECONDS, new ArrayBlockingQueue<>(1),
        new ThreadPoolExecutor.AbortPolicy());
    try {
      Future<?> first = pool.submit(() -> {
        entered.countDown();
        try { gate.await(); }
        catch (InterruptedException e) { Thread.currentThread().interrupt(); }
      });
      if (!entered.await(5, TimeUnit.SECONDS)) throw new AssertionError();
      Future<Integer> second = pool.submit(() -> 42);
      boolean rejected = false;
      try { pool.submit(() -> 99); }
      catch (RejectedExecutionException expected) { rejected = true; }
      if (!rejected) throw new AssertionError();
      gate.countDown(); first.get(5, TimeUnit.SECONDS);
      if (second.get(5, TimeUnit.SECONDS) != 42) throw new AssertionError();
      System.out.println("third task rejected; accepted tasks completed");
    } finally {
      gate.countDown(); pool.shutdownNow();
      if (!pool.awaitTermination(5, TimeUnit.SECONDS)) throw new AssertionError();
    }
  }
}
```

Run it: **java PoolRejection** · Source code is also in the ZIP bundle.

QueuePipeline.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.*;
public class QueuePipeline {
  record Item(int value, boolean stop) {}
  public static void main(String[] args) throws Exception {
    BlockingQueue<Item> queue = new ArrayBlockingQueue<>(2);
    ExecutorService pool = Executors.newFixedThreadPool(2);
    try {
      Future<Integer> consumer = pool.submit(() -> {
        int sum = 0;
        while (true) {
          Item item = queue.take();
          if (item.stop()) return sum;
          sum += item.value();
        }
      });
      Future<?> producer = pool.submit(() -> {
        for (int i = 1; i <= 100; i++) queue.put(new Item(i, false));
        queue.put(new Item(0, true));
        return null;
      });
      producer.get(5, TimeUnit.SECONDS);
      if (consumer.get(5, TimeUnit.SECONDS) != 5050) throw new AssertionError();
      System.out.println("100 items consumed; sum=5050");
    } finally {
      pool.shutdownNow();
      if (!pool.awaitTermination(5, TimeUnit.SECONDS)) throw new AssertionError();
    }
  }
}
```

Run it: **java QueuePipeline** · Source code is also in the ZIP bundle.

ContextCleanup.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.*;
public class ContextCleanup {
  static final ThreadLocal<String> CONTEXT = new ThreadLocal<>();
  public static void main(String[] args) throws Exception {
    ExecutorService pool = Executors.newSingleThreadExecutor();
    try {
      Future<?> failed = pool.submit(() -> {
        CONTEXT.set("request-A");
        try { throw new IllegalStateException("simulated request failure"); }
        finally { CONTEXT.remove(); }
      });
      try { failed.get(5, TimeUnit.SECONDS); throw new AssertionError(); }
      catch (ExecutionException e) {
        if (!(e.getCause() instanceof IllegalStateException)) throw e;
      }
      Future<Boolean> next = pool.submit(() -> CONTEXT.get() == null);
      if (!next.get(5, TimeUnit.SECONDS)) throw new AssertionError();
      System.out.println("exceptional scope cleaned; next request has no context");
    } finally {
      pool.shutdownNow();
      if (!pool.awaitTermination(5, TimeUnit.SECONDS)) throw new AssertionError();
    }
  }
}
```

Run it: **java ContextCleanup** · Source code is also in the ZIP bundle.

CompletionChain.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.*;
public class CompletionChain {
  public static void main(String[] args) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(2);
    try {
      CompletableFuture<Integer> first =
          CompletableFuture.supplyAsync(() -> 20, pool);
      CompletableFuture<Integer> second =
          CompletableFuture.supplyAsync(() -> 22, pool);
      int sum = first.thenCombine(second, Integer::sum)
          .get(5, TimeUnit.SECONDS);
      CompletableFuture<Integer> flattened = first.thenCompose(n ->
          CompletableFuture.supplyAsync(() -> n * 2, pool));
      CompletableFuture<Integer> recovered =
          CompletableFuture.<Integer>failedFuture(new IllegalStateException())
              .exceptionally(error -> 7);
      if (sum != 42 || flattened.get(5, TimeUnit.SECONDS) != 40
          || recovered.get() != 7) throw new AssertionError();
      System.out.println("combine=42; compose=40; recovery=7");
    } finally {
      pool.shutdownNow();
      if (!pool.awaitTermination(5, TimeUnit.SECONDS)) throw new AssertionError();
    }
  }
}
```

Run it: **java CompletionChain** · Source code is also in the ZIP bundle.

PeriodicFailure.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
public class PeriodicFailure {
  public static void main(String[] args) throws Exception {
    ScheduledExecutorService scheduler = Executors.newSingleThreadScheduledExecutor();
    AtomicInteger calls = new AtomicInteger();
    try {
      ScheduledFuture<?> task = scheduler.scheduleAtFixedRate(() -> {
        calls.incrementAndGet();
        throw new IllegalStateException("simulated periodic failure");
      }, 0, 10, TimeUnit.MILLISECONDS);
      try { task.get(5, TimeUnit.SECONDS); throw new AssertionError(); }
      catch (ExecutionException e) {
        if (!(e.getCause() instanceof IllegalStateException)) throw e;
      }
      if (!task.isDone() || task.isCancelled() || calls.get() != 1)
        throw new AssertionError();
      System.out.println("periodic task failed; executions=1");
    } finally {
      scheduler.shutdownNow();
      if (!scheduler.awaitTermination(5, TimeUnit.SECONDS)) throw new AssertionError();
    }
  }
}
```

Run it: **java PeriodicFailure** · Source code is also in the ZIP bundle.

AtomicTotals.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;
public class AtomicTotals {
  public static void main(String[] args) throws Exception {
    AtomicInteger count = new AtomicInteger();
    ExecutorService pool = Executors.newFixedThreadPool(4);
    try {
      Future<?>[] tasks = new Future<?>[4];
      for (int t = 0; t < tasks.length; t++) tasks[t] = pool.submit(() -> {
        for (int i = 0; i < 10_000; i++) count.incrementAndGet();
      });
      for (Future<?> task : tasks) task.get(5, TimeUnit.SECONDS);
      if (count.get() != 40_000) throw new AssertionError();
      System.out.println("atomic total=40000");
    } finally {
      pool.shutdownNow();
      if (!pool.awaitTermination(5, TimeUnit.SECONDS)) throw new AssertionError();
    }
  }
}
```

Run it: **java AtomicTotals** · Source code is also in the ZIP bundle.

LostUpdate.java

Complete Java source from the attached guide. Compiled and executed against the included assertion-driven lab.

```
import java.util.concurrent.CyclicBarrier;
public class LostUpdate {
  volatile int count;
  final CyclicBarrier gate = new CyclicBarrier(2);
  void incrementInSteps() {
    int old = count;
    try { gate.await(); }
    catch (Exception e) { throw new RuntimeException(e); }
    count = old + 1;
  }
  public static void main(String[] args) throws Exception {
    LostUpdate s = new LostUpdate();
    Thread a = new Thread(s::incrementInSteps);
    Thread b = new Thread(s::incrementInSteps);
    a.start(); b.start(); a.join(); b.join();
    if (s.count != 1) throw new AssertionError(s.count);
    System.out.println("two increments, count=1");
  }
}
```

Run it: **java LostUpdate** · Source code is also in the ZIP bundle.

Java 21+ VirtualBudget.java

The source guide marks this example as optional and untested in its original Java 17 environment. This deliverable additionally compiles and runs it on the available Java 21 runtime. It demonstrates that cheap threads do not remove the need to bound shared resources.

```
import java.util.*;
import java.util.concurrent.*;
public class VirtualBudget {
  public static void main(String[] args) throws Exception {
    Semaphore permits = new Semaphore(4);
    List<Future<Integer>> tasks = new ArrayList<>();
    try (ExecutorService executor =
        Executors.newVirtualThreadPerTaskExecutor()) {
      for (int i = 0; i < 20; i++) {
        final int value = i;
        tasks.add(executor.submit(() -> {
          if (!Thread.currentThread().isVirtual()) throw new AssertionError();
          permits.acquire();
          try {
            Thread.sleep(10); // synthetic wait, not a service benchmark
            return value * 2;
          } finally { permits.release(); }
        }));
      }
      int total = 0;
      for (Future<Integer> task : tasks)
        total += task.get(5, TimeUnit.SECONDS);
      if (total != 380 || permits.availablePermits() != 4)
        throw new AssertionError();
      System.out.println("virtual task sum=380");
    }
  }
}
```

Compile with **javac --release 21 VirtualBudget.java**. Expected output: virtual task sum=380.

Two weeks, one clear objective.

DAY	TOPIC	ACTIVITY
01	Foundations	Q001–Q005: tasks, processes and concurrency
02	Lifecycle	Q006–Q010 + Lab 01
03	Scheduling	Q011–Q015; read a thread dump
04	Cancellation	Q016–Q020 + Lab 03
05	Shared state	Q021–Q025 + Labs 09/10
06	Waiting	Q026–Q030 + Lab 02
07	Deadlocks	Q031–Q035; draw resource cycles
08	Executors	Q036–Q040 + Lab 04
09	Admission	Q041–Q045; calculate bounded capacity
10	Coordination	Q046–Q050 + Lab 05
11	Async stages	Q051–Q055 + Lab 07
12	Scheduler + context	Q056–Q065 + Labs 06/08
13	Modern & coding	Q066–Q075; try virtual-thread example
14	Senior mock	Q076–Q080; explain every tradeoff

Keep a one-page log each day: what the JDK guarantees, one code trace, one failure case, and the evidence you would collect.

Can you defend it?

10′ Lifecycle & cancellation

Explain start vs run, join, interrupt, cancel(true) and how you prove termination.

10′ Shared state

Walk through volatile count++, an atomic counter, one lock and one immutable snapshot.

10′ Executors & overload

Describe the workers → queue → grow/reject admission path, then a pool starvation cycle.

10′ Modern concurrency

Compare Futures, CompletableFuture and virtual threads, including explicit resource limits.

10′ Production incident

Use thread dumps, queue wait, dependency metrics, cancellation and error rate to justify a safe fix.

Self-score (0–3): 0 = incorrect; 1 = core idea; 2 = clear example and limitation; 3 = complete invariant, failure boundary and diagnostic evidence.